

面经	2
面试组成（三点）	2
公司面经	2
华为	2
八股干货	3
手撕代码	4
异步FIFO	5
分频器（偶数分频、奇数分频、分数分频）	8
边沿检测(上升沿、下降沿、双边沿)	11
序列检测(状态机)	12
半加器、全加器(单bit、多bit、超前进位)	15
仲裁器(固定优先级)	18
仲裁器(轮询)	19
格雷码与二进制转换	22
门控时钟(如何唤醒)	24
无毛刺时钟切换电路	26
串并转换	28
流水线握手、反压	29
异步复位同步释放	31
模3检测	33
异步电路多bit同步-握手	34
乒乓操作	34
静态时序分析	41
异步电路CDC	42
两级同步器：	42
握手协议：	44
脉冲同步器	46
异步FIFO	47
DMUX同步	47
一文掌握DC综合流程	48
如何设计时序收敛的RTL代码	49
verilog  电路	50

面经

面试组成（三点）

- ① 智力题：包括但不限于逻辑推理、数字推理。
- ② 数字IC相关八股知识点
 - ① 同步复位与异步复位的优缺点；
 - ② 静态时序分析、建立保持时间的计算（重点常考）；
 - ③ 分频电路，奇数分频、任意小数分频、偶数分频（可能会手撕代码）；
 - ④ 通信协议：I2C、SPI、AXI、AMBA等；
 - ⑤ 经典的阻塞非阻塞赋值的定义、区别、电路；
 - ⑥ 异步FIFO的设计原理，简单的代码；
 - ⑦ 跨时钟域的相关问题与电路；
 - ⑧ 低功耗设计（功耗组成和设计方法设计各个层面）；
 - ⑨ 仲裁器、手搭全加器、消抖电路、无毛刺时钟切换电路；
 - ⑩ 同步FIFO、异步FIFO；
 - ⑪ 格雷码、独热码等编码方式；
 - ⑫ 三段式状态机，Mealy、Moore；
 - ⑬ 乒乓操作、流水线，流水线反压。
- ③ 项目细节
 - ① 创新点
 - ② 难点以及怎么解决、为什么这么解决
 - ③ 项目PPA指标

公司面经

华为

A一面：

- ① 芯片选型关注哪些参数；
- ② 手撕代码计数器，高电平复位，要求有一个计数清零端，计满保持；
- ③ 验证方法懂吗？提升覆盖率的方法懂吗？
- ④ FPGA开发遇到的问题；
- ⑤ 低功耗方法有哪些；
- ⑥ 毛刺怎么来的；
- ⑦ 讲解下门控时钟怎么实现；
- ⑧ 项目介绍；
- ⑨ 计算-6.25十进制转8bit二进制；

岗位：芯片与器件工程师

B一面：

- ① 手撕代码（交通灯）；

八股干货

芯日记

芯日记

芯日记

手撕代码

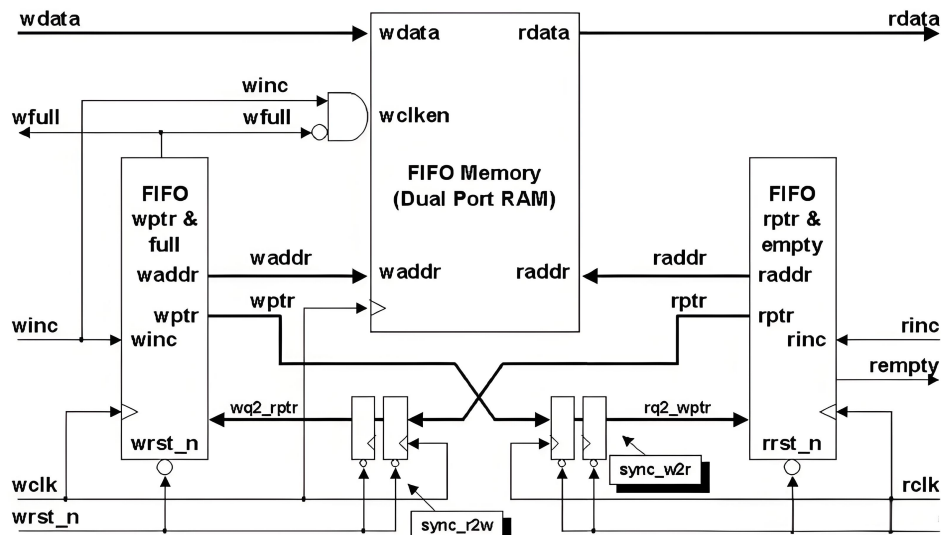
秋招必刷15道verilog题（+ 芯日记）

- ✓ 异步FIFO
- ✓ 分频电路(偶数、奇数、分数分频)
- ✓ 边沿检测(上升沿、下降沿、双边沿)
- ✓ 序列检测(状态机)
- ✓ 半加器、全加器(单bit、多bit、超前进位)
- ✓ 仲裁器(固定优先级、轮询)
- ✓ 格雷码与二进制转换
- ✓ 门控时钟(如何唤醒)
- ✓ 无毛刺时钟切换电路
- ✓ 串并转换
- ✓ 流水线握手、反压
- ✓ 同步释放异步复位
- ✓ 模3检测
- ✓ 异步电路多bit同步-握手
- ✓ 乒乓操作

异步FIFO

1、设计思路

关键的点：读写地址计算、空满状态产生、读写控制信号的生成。



(1) 读控制 (pop)、写控制 (push) 的生成：

当外部输入的winc=1且full=0时，就可以对fifo进行写操作，即 $push = !full \ \& \ winc$;

当外部输入的rinc=1且empty=0时，就可以对fifo进行读操作，即 $pop = (!empty) \ \& \ rd_en$;

(2) 读地址 (raddr)、写地址 (waddr)：

时钟上升沿到来且读使能 (pop) 或者写使能 (push) 有效，读地址 (读指针) +1，写地址 (写指针) +1。

对应到verilog代码为：

```
always@(posedge clk)
```

```
if(reset)
```

```
    waddr<=5'b0;
```

```
else if(push)
```

```
    waddr<=waddr+1;
```

```
always@(posedge clk)
```

```
if(reset)
```

```
    raddr<=5'b0;
```

```
else if(pop)
```

```
    raddr<=raddr+1;
```

(3) 空满状态判断

空满状态的判断需要计算出读写地址的差值。使用一个数据计数器cnt来计算当前FIFO中存储的数据个数，根据计数器cnt的值来判断FIFO的空信号empty与满信号full。如果计数器cnt=0，表示FIFO中当前存储的数据个数为0，则拉高空信号empty；如果计数器cnt=FIFO_DEPTH (FIFO深度)，则表示拉高满信号full。那么，在异步FIFO的设计中，是否也可以使用计数器的方式判断空满信号呢？在异步FIFO的设计中，我们不用计数器的方式来判断空满，而是将读写指针地址的位宽增加一位冗余，用于判断空满信号。但因为是异步的，所以需要进行跨时钟域传输读写地址。对于多

比特的数据，并且FIFO的深度一般都是2的N次方，因此最常用的同步方法就是使用格雷码，这样相邻两位就只有1bit发生变化。

二进制转换成格雷码的方法：将二进制右移移位后，和右移前的数值进行异或即可。

//读地址从二进制转变成格雷码，已省去位宽

```
assign raddr_gray<=raddr^(raddr>>1);
assign waddr_gray<=waddr^(waddr>>1);
```

转换成格雷码后，就可以通过打两拍的方法进行同步，降低亚稳态发生的概率。

//gray write pointer sync

```
always @(posedge r_clk or negedge rd_rst_n) begin
```

```
    if(!rd_rst_n) begin
```

```
        waddr_gray_w2r_1 <= 'd0;
```

```
        waddr_gray_w2r_2 <= 'd0;
```

```
    end
```

```
    else begin
```

```
        waddr_gray_w2r_1 <= waddr_gray;
```

```
        waddr_gray_w2r_2 <= waddr_gray_w2r_1;
```

```
    end
```

```
end
```

//gray read pointer sync

```
always @(posedge w_clk or negedge wr_rst_n) begin
```

```
    if(!wr_rst_n) begin
```

```
        raddr_gray_r2w_1 <= 'd0;
```

```
        raddr_gray_r2w_2 <= 'd0;
```

```
    end
```

```
    else begin
```

```
        raddr_gray_r2w_1 <= raddr_gray;
```

```
        raddr_gray_r2w_2 <= raddr_gray_r2w_1;
```

```
    end
```

```
end
```

注意⚠：这里用的都是格雷码，用格雷码来判断空满是不是和二进制一样呢？答案是空信号一样，满信号不一样。

- **判断空**：将写指针同步到读时钟域，与读指针进行比较，每一位都完全相同才判断为读空；
- **判断满**：将读指针同步到写时钟域，与写指针进行比较，最高两位不相等，其余各位完全相同则写满。

//full and empty

```
assign full = (waddr_gray == {~raddr_gray_r2w_2[N:N-1], rd_ptr_gray_r2w_2[N-2:0]}) ;
```

```
assign empty = (rd_ptr_gray == wr_ptr_gray_w2r_2);
```

这里有三个问题可以研究🤔：

✅1、为什么要把读（写）指针同步到写（读）时钟域来判断满（空）？

同步到写时钟域：

读指针同步到写时钟域需要2个cycle，在经过2个cycle之后，可能原来的读指针会增加，也就是说同步后的读指针一定是小于等于原来的读指针的。

写满判断：写指针超过 同步后的读指针一圈，但是由于有2个周期的延迟，原来的读指针是大于等于同步后的读指针的，所以实际上这个时候写指针其实是没有超过读指针一圈的，也就是说这种情况是“**假满**”。但这并没有逻辑错误，只是少存了2笔数据。

同步到读时钟域：

写指针同步到读时钟域需要2个cycle，在经过2个cycle之后，可能原来的写指针会增加，也就是说同步后的写指针一定是小于等于原来的写指针的。

读空判断：也就是读指针追上了同步后的指针。但是原来的写指针是大于等于同步后的写指针的，所以实际上这个时候读指针其实还没有追上写指针，也就是说这种情况是“**假空**”。也就是说这个FIFO，在读到还剩2个数据的时候就报了“空”，这并不会造成功能错误，只会造成性能损失。这就是异步fifo的保守设计。

✅2、为什么格雷码判断满，要最高两位不相等呢？

① wptr和同步过来的rptr的MSB不相等，因为wptr必须比rptr多折回一次。

② wptr与rptr的次高位不相等，如上图位置7和位置15，转化为二进制对应的是0111和1111，MSB不同说明多折回一次，111相同代表同一位置。

✅3、将地址转变成格雷码的形式后采用打两拍的方法意味着是慢到快时钟域，如果读写时钟频率不同呢？

(1) 读慢写快

进行写满判断的时候需要将读指针同步到写时钟域，因为读慢写快，所以不会有读指针遗漏，同步消耗时钟周期，所以同步后的读指针滞后（小于等于）当前读地址，所以可能写满会提前产生，并非真写满。

进行读空判断的时候需要将写指针同步到读指针，因为读慢写快，所以当读时钟同步写指针的时候，必然会漏掉一部分写指针，我们不用关心那到底会漏掉哪些写指针，我们在乎的是漏掉的指针会对FIFO的读空产生影响吗？比如写指针从0写到10，期间读时钟域只同步捕捉到了3、5、8这三个写指针而漏掉了其他指针。当同步到8这个写指针时，真实的写指针可能已经写到10，相当于在读时钟域还没来得及觉察的情况下，写时钟域可能写了数据到FIFO去，这样在判断它是不是空的时候会出现不是真正空的情况，漏掉的指针也没有对FIFO的逻辑操作产生影响。

(2) 读快写慢

进行读空判断的时候需要将写指针同步到读指针，因为读快写慢，所以不会有写指针遗漏，同步消耗时钟周期，所以同步后的写指针滞后（小于等于）当前写地址，所以可能读空会提前产生，并非真读空。

进行写满判断的时候需要将读指针同步到写时钟域，因为读快写慢，所以当写时钟同步读指针的时候，必然会漏掉一部分读指针，我们不用关心那到底会漏掉哪些读指针，我们在乎的是漏掉的指针会对FIFO的写满产生影响吗？比如读指针从0读到10，期间写时钟域只同步捕捉到了3、5、8这三个读指针而漏掉了其他指针。当同步到8这个读指针时，真实的读指针可能已经读到10，相当于在写时钟域还没来得及觉察的情况下，读时钟域可能从FIFO读了数据出来，这样在判断它是不是满的时候会出现不是真正满的情况，漏掉的指针也没有对FIFO的逻辑操作产生影响。

分频器（偶数分频、奇数分频、分数分频）

I. 偶数分频

偶数分频是最简单的一种，比如2、4、6、8分频等。

其原理是利用计数器对时钟上升沿进行计数，在特定计数值时对输出时钟进行翻转。

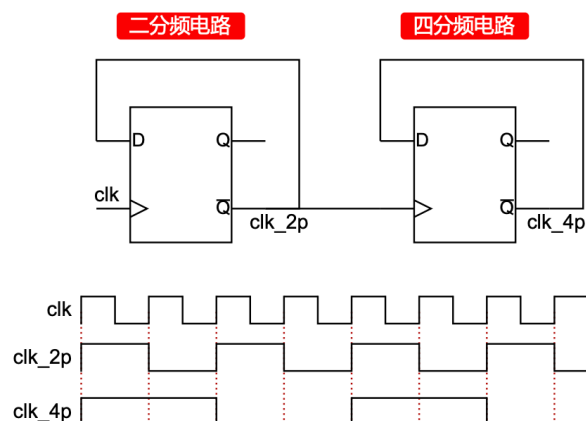
以6分频为例（ $N=6$ ），我们需要一个计数器，当计数值达到一半（ $N/2 - 1$ ），将输出时钟翻转。

细节代码实现（50%占空比）

```
// 计数器：从0计数到 (N/2 - 1)
reg [31:0] cnt;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cnt <= 0;
    end else begin
        if (cnt == (N/2 - 1)) begin
            cnt <= 0;
        end else begin
            cnt <= cnt + 1;
        end
    end
end
// 在计数器为0时翻转时钟，实现50%占空比
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        clk_out <= 1'b0;
    end else
        if (cnt == (N/2 - 1))// 当计数到最大值时翻转
            clk_out <= ~clk_out;
end
```

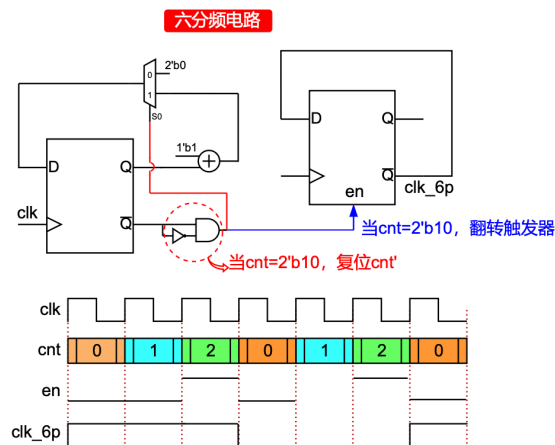
偶数分频的代码很好写，但是你能否手撕六分频的电路呢🤔？

2分频电路估计大家都能直接画出来，一个寄存器的Q取反接到D端，Q非就是2分频后的时钟，电路如下：



同理，四分频、八分频可以依次类推。但是六分频并不是2的N次幂，不能简单的依次类推。

六分频的重点是有一个计数的cnt，在等于2'b10的时候要复位成2'b00。因此需要1个MUX来判断当前的cnt是不是等于2'b10，判断方法也很简单就是 $en = \sim cnt[0] \& cnt[1]$ ；当en等于1的时候有两个操作，一个是复位cnt，另一个是翻转输出时钟，那根据这些信息就可以画出如下的六分频电路图。



红色圆圈内就是判断cnt是不是等于2'b10的逻辑，当等于2'b10的时候，进行MUX选择2'b00(cnt复位)和翻转时钟。

II. 奇数分频

奇数分频（如3、5、7分频）比偶数分频复杂，因为无法通过直接在原时钟的单个边沿（如上升沿）计数来简单得到一个50%占空比的时钟。核心思想是产生两个相位差180度的信号，然后将它们组合。

以5分频（N=5）为例：

1. 创建两个分频信号：

使用两个计数器，都从0计数到4（N-1）。

信号A (clk_p)：在时钟上升沿变化。在计数器计到(N-1)/2和N-1时翻转。对于N=5，就是在计到2和4时翻转。

信号B (clk_n)：在时钟下降沿变化。使用同样的翻转规则，即在下降沿计数到2和4时翻转。

最后： $clk_out = clk_p \mid clk_n$ ，生成最终时钟。

代码实现（50%占空比）

// 上升沿部分

```
always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) begin
```

```
        cnt_p <= 0;
```

```
        clk_p <= 1'b0;
```

```
    end else begin
```

```
        if (cnt_p == (N-1)) begin// 计数器逻辑
```

```
            cnt_p <= 0;
```

```
        end else begin
```

```
            cnt_p <= cnt_p + 1;
```

```
        end
```

```
        if (cnt_p == (N-1)/2 || cnt_p == (N-1)) begin// 时钟生成逻辑：在 (N-1)/2 和 N-1 时翻转
```

```
            clk_p <= ~clk_p;
```

```
        end
```

```
    end
```

```

end
// 下降沿部分
always @(negedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cnt_n <= 0;
        clk_n <= 1'b0;
    end else begin
        if (cnt_n == (N-1)) begin
            cnt_n <= 0;
        end else begin
            cnt_n <= cnt_n + 1;
        end
        if (cnt_n == (N-1)/2 || cnt_n == (N-1)) begin
            clk_n <= ~clk_n;
        end
    end
end
end
assign clk_out = clk_p | clk_n; // 或者使用 clk_p & clk_n, 取决于初始相位

```

代码分析：

cnt_p和clk_p在时钟上升沿工作。clk_p在计数值达到(N-1)/2（即2）和N-1（即4）时翻转。

cnt_n和clk_n在时钟下降沿工作，逻辑与上升沿部分完全一致。

最终，clk_p和clk_n是两个占空比都不是50%，但相位相差半个原时钟周期的5分频信号。将它们进行“或”操作后，就得到了50%占空比的5分频时钟clk_out。

III. 分数分频

这类分频通常用于需要特定频率比的场景，例如2.5分频、3.5分频等。

电路原理（以2.5分频为例）

2.5分频意味着输出时钟周期是输入时钟周期的2.5倍。

- 对于N.5分频（如2.5），可以交替进行N分频和N+1分频。
- 目标：在2个输出周期内，总共消耗5个输入时钟周期（因为 $2 * 2.5 = 5$ ）。
 - 第一个输出周期持续3个clk周期（3分频）。
 - 第二个输出周期持续2个clk周期（2分频）。
 - 如此循环，平均周期就是 $(3+2)/2 = 2.5$ 。

代码实现（2.5分频）

```

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        clk_out <= 1'b0;
        cnt <= 1'b0;
    end else begin
        // 根据cnt的值，决定本次是2分频还是3分频
        if (cnt == 1'b0) begin
            // 3分频模式
            // 这里用一个简单的状态机实现
            // 例如，在计满2个周期后翻转（实际实现可能需要一个更精细的计数器）
            // 简化实现：使用一个模3计数器

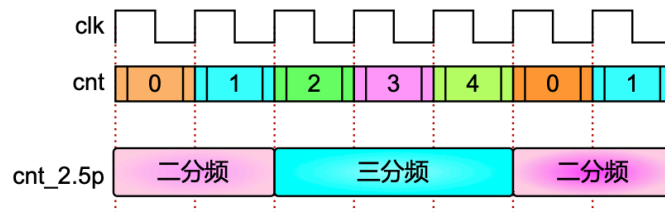
```

```

end else begin
    // 2分频模式
    // 简化实现：使用一个模2计数器
end
cnt <= ~cnt; // 每个输出周期结束后切换模式
end
end

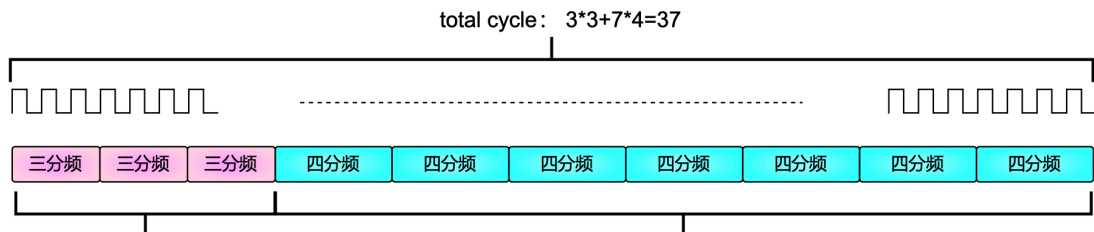
```

如下图所示，2.5分频只需要1个二分频和1个三分频交替出现就是2.5分频。



再举另一个例子，3.7分频。假设需要三分频 x 个，四分频 y 个，满足方程 $3x+4y=37$ ； $x+y=10$ ；得到 $x=3$ ， $y=4$ 。

也就是说需要在37个原始时钟周期内，前9个时钟周期(cnt从0到8)，输出3个三分频时钟，在后28个时钟周期(cnt从9到36)输出7个四分频时钟即可实现3.7的分数分频。



边沿检测(上升沿、下降沿、双边沿)

1、为什么需要边沿检测？

在同步数字系统中，很多操作（如状态机的状态转移、数据的锁存、计数器的使能等）都需要在某个特定事件发生的瞬间被触发。这个“瞬间”就是信号边沿。

- **按键消抖后的动作触发：**当你按下物理按键，经过消抖电路后得到一个稳定的高电平。我们通常希望在按键刚刚被按下（上升沿）或刚刚被释放（下降沿）的瞬间执行一次操作，而不是在整个按键按下的期间内重复执行。这就需要检测边沿。
- **跨时钟域信号同步：**当信号从一个时钟域传递到另一个时钟域时，需要检测信号的变化，以便在目标时钟域进行安全可靠的采样。
- **状态机触发：**状态机的跳转条件常常是某个信号的边沿，而不是电平。

2、基本原理

边沿检测的核心思想是：通过将信号延迟一个时钟周期，然后将原始信号与延迟后的信号进行比较。

1. 上升沿检测: $\text{pos_edge} = \text{sig} \ \& \ \sim\text{sig_dly}$
2. 下降沿检测: $\text{neg_edge} = \sim\text{sig} \ \& \ \text{sig_dly}$
3. 双边沿检测: $\text{double_edge} = \text{sig} \ ^\wedge \ \text{sig_dly}$ (异或运算: 相同为0, 不同为1)

⚠️ 边沿检测电路 (尤其是用纯数字逻辑搭建的) 虽然原理简单, 但在实际硬件实现中, 如果考虑不周, 会引入几个非常重要的风险, 主要可以归结为三大类: **亚稳态**、**毛刺**和**时序问题**。

1. 亚稳态, 这是异步电路设计中最经典、最危险的问题。

但使用两级触发器串联, 基本上可以将风险降低到一个可以接受的水平。

2. 毛刺, 如果你用一个简单的“与门+非门+延迟线”这种异步方案来构建边沿检测, 毛刺风险很高。边沿检测电路对毛刺极其敏感! 它无法区分这是一个真正的信号跳变, 还是一个由逻辑竞争产生的毛刺。它会忠实地将毛刺识别为一个“边沿事件”, 并输出一个错误的脉冲。这会导致计数器多计数、状态机错误跳转等严重问题。

解决方法:

2.1. 同步设计: 优先采用基于同步时钟和D触发器的边沿检测电路。这种电路对组合逻辑毛刺的免疫力强得多, 因为毛刺通常无法满足触发器的建立和保持时间。

2.2. 滤波: 对于低速的异步输入 (如按键), 可以在信号进入FPGA/CPLD之前, 先通过一个简单的RC低通滤波器进行硬件滤波, 滤除高频毛刺。

3. 时序与性能问题

3.1脉冲宽度: 边沿检测输出的脉冲宽度是一个时钟周期。如果你的系统时钟非常快 (比如100MHz), 那么这个脉冲只有10ns宽。这个脉冲可能需要驱动多个模块, 或者需要穿越很长的布线路径, 这么窄的脉冲可能在到达目的地之前就“消失”了 (由于路径延迟)。这被称为同步脉冲的缩小。

3.2时钟频率限制: 边沿检测电路要求输入信号的变化间隔必须大于至少一个时钟周期。如果输入信号的变化太快 (例如, 两个连续上升沿之间的间隔小于一个时钟周期), 那么第二个边沿将无法被检测到, 导致漏检。

🤔 为了规避这些风险, 在现代数字设计 (FPGA/ASIC) 中, 构建一个稳健的边沿检测电路应遵循以下原则:

1. 永远对异步输入进行同步: 使用至少两级触发器组成的同步器来处理所有来自外部或其它时钟域的输入信号。
2. 采用同步电路设计: 使用时钟驱动的触发器来设计边沿检测电路, 而不是依赖组合逻辑和延迟线。例如, 通过缓存上一个时钟周期的信号值, 与当前值进行比较来产生边沿脉冲。
3. 考虑脉冲宽度: 在高速系统中, 评估一个时钟周期的脉冲是否足够“宽”以被后续电路正确捕获。如果不够, 可能需要适当降低时钟频率或对脉冲进行展宽。
4. 理解输入信号的特性: 了解待检测信号的最大频率, 确保你的系统时钟频率远高于它, 以避免漏检。

序列检测(状态机)

序列检测电路是数字电路中用于检测一组特定的二进制码 (序列) 是否在输入数据流中出现的电路。它在通信 (如帧头检测)、数据校验 (如CRC)、安全控制和模式识别等领域有广泛应用。

序列检测一般有两种

1、**重叠模式检测**: 当检测到一个序列后, 检测器的最后一位或几位可以作为下一个序列的起始位。

示例：检测 "1011"。输入流 ...101011...。

第一个序列：10101 (检测到第1个 "1011")

第二个序列：101011 (检测到第2个 "1011")。注意，第一个序列的结尾 "11" 被第二个序列重叠使用了。

2、**非重叠模式检测**：当检测到一个序列后，电路回到初始状态，下一个序列必须从头开始检测。

示例：检测 "1011"。输入流 ...101011...。

第一个序列：10101 (检测到)

检测完成后，电路复位。接下来的 01 无法构成序列的开头，所以第二个 "1011" 不会被检测到，因为它是重叠的。

✅ 用状态机实现“1011”序列检测

// 状态定义：使用独热码 (One-Hot) 或二进制码

// 这里使用独热码，更清晰且易于综合

localparam S0 = 3'b000;

localparam S1 = 3'b001;

localparam S2 = 3'b010;

localparam S3 = 3'b100;

// S4状态不需要，因为检测成功是S3在特定输入下产生的输出

reg [2:0] current_state;

reg [2:0] next_state;

// 状态寄存器（时序逻辑）

always @(posedge clk or negedge rst_n) begin

if (!rst_n)

current_state <= S0;

else

current_state <= next_state;

end

// 下一状态逻辑和输出逻辑（组合逻辑）

always @(*) begin

// 默认值

next_state = S0;

det_out = 1'b0;

case (current_state)

S0: begin

if (data_in == 1'b1)

next_state = S1;

else

next_state = S0;

end

S1: begin

if (data_in == 1'b0)

next_state = S2;

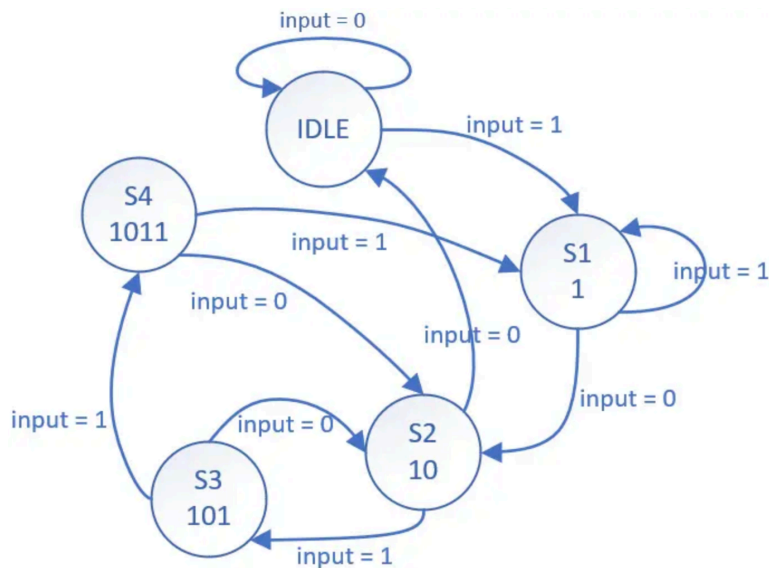
else

```

    next_state = S1; // 重叠, 新序列开始
end
S2: begin
    if (data_in == 1'b1)
        next_state = S3;
    else
        next_state = S0; // "100"失败, 回到起点
    end
end
S3: begin
    if (data_in == 1'b1) begin
        // 检测成功! 输出脉冲, 并进入下一个状态 (重叠)
        det_out = 1'b1;
        next_state = S1; // 重叠: 最后的'1'作为新序列的开始
        // 如果是非重叠, 则 next_state = S0;
    end else begin
        next_state = S2; // "1010"失败, 但"10"有效, 回到S2
    end
end
end
default: next_state = S0;
endcase
end

```

状态机的代码比较固定, 区别就是状态转移图不同, 因此画状态转移图也是面试常考的问题。下图是“1011”序列的状态转移图。



✅ 另一种更简单的方法是用移位寄存器实现序列检测。

// 定义一个4位的移位寄存器, 用于检测4位序列 "1011"

```

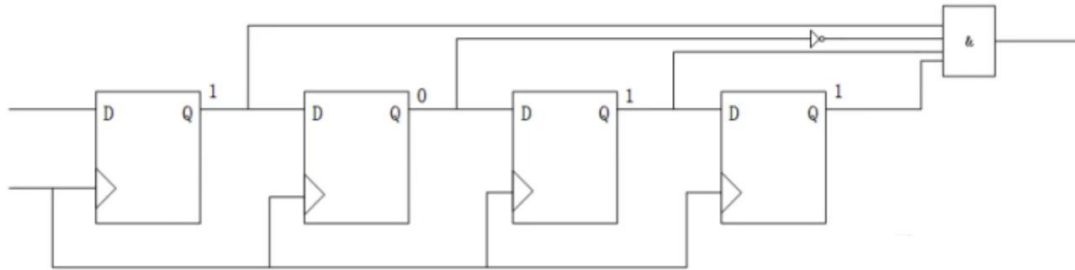
reg [3:0] shift_reg;
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        shift_reg <= 4'b0;
    else

```

```

    shift_reg <= {shift_reg[2:0], data_in}; // 左移或右移均可，这里为左移
end
always @(*) begin
    if (shift_reg == 4'b1011) // 要检测的序列
        det_out = 1'b1;
    else
        det_out = 1'b0;
end

```



移位寄存器的实现电路图如上图。

半加器、全加器(单bit、多bit、超前进位)

半加器和全加器的区别在于，是否有进位输入端，可以直观地理解为，半加器是两个一比特相加，而全加器是三个一比特相加，输出结果和进位信号。

根据二进制加法规则，我们可以得到半加器和全加器的真值表：

A	B	C_out	Sum	计算 (A+B)
0	0	0	0	0 + 0 = 0
0	1	0	1	0 + 1 = 1
1	0	0	1	1 + 0 = 1
1	1	1	0	1 + 1 = 10 (二进制)

A	B	C_in	C_out	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

根据真值表，我们可以直接写出输出表达式。

半加器

- $\text{Sum} = A \oplus B$ (异或运算)
- $\text{C_out} = A \cdot B$ (与运算)

全加器：

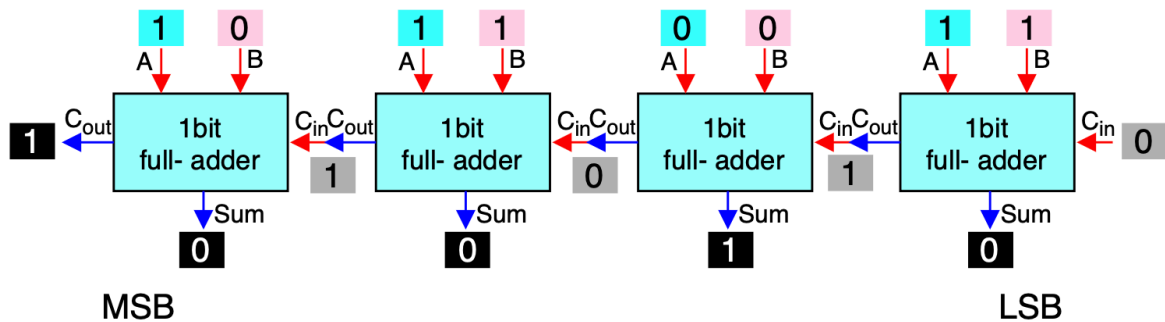
- $\text{Sum} = A \oplus B \oplus \text{C_in}$
- $\text{C_out} = (A \cdot B) + (B \cdot \text{C_in}) + (A \cdot \text{C_in})$

电路图就不赘述了，根据逻辑表达式应该很好画。

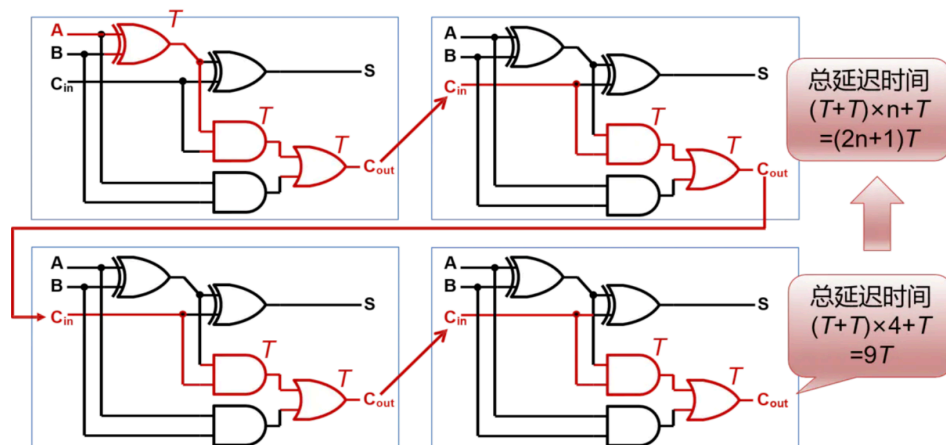
下面思考这两个问题 多bit加法器怎么构建？

行波进位的方法是最容易想到的，如下图。

计算：1101+0101=1_0010



先分析下全加器的逻辑表达式。C_in经过一堆组合逻辑生成C_out，假设这个组合逻辑的延时是 T_d ，并且同时上一级的C_out又是前一级的C_in，那如果这个4bit的全加器要在一个时钟周期内出结果，也就是说LSB的全加器的C_in要生成MSB的C_out所经过的组合逻辑产生的延时就是 $4 \cdot T_d$ 。如果时钟周期足够大，自然也是没问题的。



那如果时钟周期不够，又想在一个周期内出结果，怎么办呢？那就是用超前进位的方法。

核心思想就是 **并行计算所有位的进位**，而不是等待前一级的进位结果。它通过分析加数和被加数的每一位，直接计算出每一位的进位信号。

回顾全加器的进位公式：

$$\text{C}_{i+1} = (\text{A}_i \& \text{B}_i) \mid ((\text{A}_i \wedge \text{B}_i) \& \text{C}_i)$$

我们定义两个中间信号：

生成信号： $G_i = A_i \& B_i$ ，表示本位自身会产生一个进位（无论有无进位输入）。

传播信号： $P_i = A_i \wedge B_i$ ，表示本位会将低位的进位传递到高位（如果 A_i 和 B_i 只有一个为1，则和等于进位输入）。

那么，进位公式可以重写为： $C_{i+1} = G_i \mid (P_i \& C_i)$

现在，我们可以展开每一位的进位：

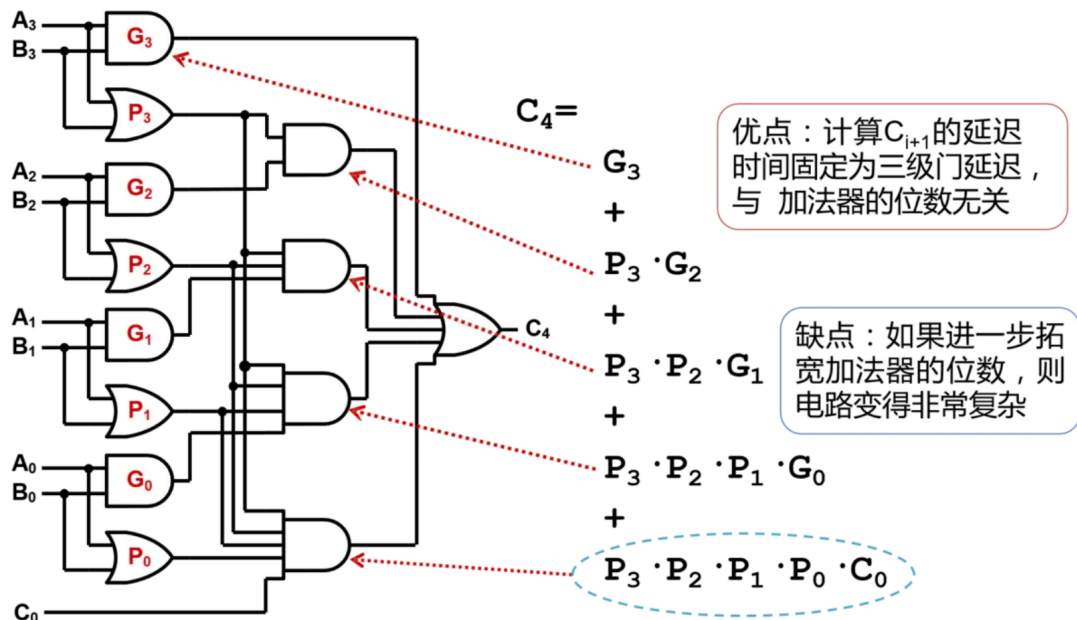
$$C_1 = G_0 \mid (P_0 \& C_0)$$

$$C_2 = G_1 \mid (P_1 \& C_1) = G_1 \mid (P_1 \& G_0) \mid (P_1 \& P_0 \& C_0)$$

$$C_3 = G_2 \mid (P_2 \& C_2) = G_2 \mid (P_2 \& G_1) \mid (P_2 \& P_1 \& G_0) \mid (P_2 \& P_1 \& P_0 \& C_0)$$

$$C_4 = \dots$$

可以看到， $C_2, C_3, C_4 \dots$ 都可以直接由 C_0 （默认为0）， G_i, P_i 计算得出，而不需要依赖前一级的进位结果。这就是“超前进位”的精髓。



优点：速度非常快。所有进位几乎同时产生，关键路径延迟与位宽的对数成正比（ $O(\log N)$ ）。

缺点：电路复杂，面积大。进位逻辑随着位宽增加而急剧膨胀。对于非常大的位宽（如64位），电路会变得不切实际。

😓 那想要实现64bit的加法，1个周期是实现不了了吗？这个需要根据实际项目权衡，目前用的较多的是 **分组超前进位加法器**：将大的加法器分成几个小组，组内使用超前进位，组间可以使用行波进位或更高层的超前进位。

例如，一个16位加法器可以：

组内并行：分成4个4位超前进位加法器模块。

组间串行：组与组之间采用行波进位。这比纯粹的16位行波进位快，但比纯粹的16位超前进位面积小。

组间并行：再使用一层超前进位逻辑来并行产生小组之间的进位。这提供了更好的性能。

现代EDA综合工具（如Design Compiler, Vivado）在综合 $A + B$ 这样的行为级代码时，会自动根据时序和面积约束，选择最优的加法器结构（通常是分组超前进位或其他更先进的结构）。

✅ 总结下就是：

加法器类型	原理	延迟	面积	应用场景
行波进位	进位逐级传递	$O(N)$	小	对速度要求不高的低成本、低功耗设计
超前进位	进位并行计算	$O(\log N)$	大	对速度要求极高的关键路径
分组超前进位	组内并行，组间并行/串行	$O(\log N)$	中等	实际工程中最常用的折中方案

仲裁器(固定优先级)

首先仲裁器是做什么的？➡️ 用于管理多个主设备对同一共享资源（如总线、存储器）的访问请求，确保在任何时刻只有一个主设备能获得访问权限，避免冲突。

固定优先级，顾名思义就是当前的多个slave的优先级是固定的，也就是说当有多个请求冲突时，高优先级的slave会被一直仲裁。

	A 🧐	B 🧐	C 🧐	D 🧐	
优先级	0(最低优先级)	1	2	3(最高优先级)	
请求	0	0	0	1(✅)	第1轮仲裁
	0(最低优先级)	1	2	3(最高优先级)	
	1	1	0	1(✅)	第2轮仲裁
	0(最低优先级)	1	2	3(最高优先级)	
	1	1(✅)	0	0	第3轮仲裁
	0(最低优先级)	1	2	3(最高优先级)	
	1(✅)	0	0	0	第4轮仲裁

如上图所示，有4个slave(A、B、C、D)，优先级分别是0，1，2，3(由低到高)，当只有1个请求的时候，只仲裁请求的即可。(如第1轮仲裁)。当产生多个请求的时候，需要按照优先级顺序进行仲裁，在每轮仲裁的过程中优先级都保持不变(如第2轮仲裁，A、B、D都有请求，但优先级最高的是D，所以优先仲裁D)，以此类推后面几轮的仲裁。

🧐 上图就是固定优先级仲裁器的核心思想。接下来我们分析下实现！

首先看下仲裁器的输入输出是什么，有什么特点。

输入就是一堆请求，假设现在有8个slave，那输入就是8bit的data，相应bit位为1表示当前slave有请求。输出对应也是8bit，但是只能有1bit是1，因为仲裁只有1个，那也就是one-hot码。

我们假设 当前的输入请求是1001_0010，仲裁结果就是slave1，实际上就是找leading-one。

两种方法：

1、直接用casez写，如下：

```
casez (req[7:0])
```

```

8'b???????1: begin grant = 8'b00000001; end
8'b???????10: begin grant = 8'b00000010; end
8'b???????100: begin grant = 8'b00000100; end
8'b?????1000: begin grant = 8'b00001000; end
8'b???10000: begin grant = 8'b00010000; end
8'b??100000: begin grant = 8'b00100000; end
8'b?1000000: begin grant = 8'b01000000; end
8'b10000000: begin grant = 8'b10000000; end
default: begin grant = 8'b00000000; end // 全0情况
endcase

```

2、one-hot写法

```
grant = req & ~(req - 1'b1); // 一个数和它的补码相与，得到one-hot码
```

更通俗易懂的理解：将req减1，这样req序列中从低位开始，为0的bit位就会因为不够减向高位借位，当前位就变为1，直到遇到序列中第一个为1的bit位，其因为低位的借位变为0，更高位则保持不变。再将得到的新req序列进行按位~，此时原req序列中第一个1仍然为1，再和原req进行按位&操作，即可得到由低到高的第一个1的位置。

仲裁器(轮询)

“轮询”顾名思义就是轮流询问(仲裁)的意思，表示当前所有的请求者都有可能成为优先级最高的那一个，以确保“公平”。

如下图所示，有4个slave(A、B、C、D)，初始优先级分别是0，1，2，3(由低到高)，当只有1个请求的时候，只仲裁请求的即可，并且将当前slave C的优先级置0，左边的slave优先级置最高，从右往左优先级依次递减(如第1轮仲裁)。

当产生多个请求的时候，需要按照优先级顺序进行仲裁，并且重置优先级(如第2轮仲裁，A、B、D都有请求，但优先级最高的是B，所以优先仲裁B，并且将slave B的优先级置0，slave A的优先级置最高，slave D的优先级次高)，以此类推后面几轮的仲裁。

	A 🧑	B 🧑	C 🧑	D 🧑	
优先级	0(最低优先级)	1	2	3(最高优先级)	
请求	0	0	1(✓)	0	第1轮仲裁
	2	3(最高优先级)	0(优先级置0)	1	第2轮仲裁
	1	1(✓)	0	1	
	3(最高优先级)	0(优先级置0)	1	2	第3轮仲裁
	1(✓)	0	0	1	
	0(优先级置0)	1	2	3(最高优先级)	第4轮仲裁
	0	0	0	1(✓)	

上图就是轮询仲裁器的核心思想。接下来我们分析下实现 🤔！

首先看下仲裁器的输入输出是什么，有什么特点。

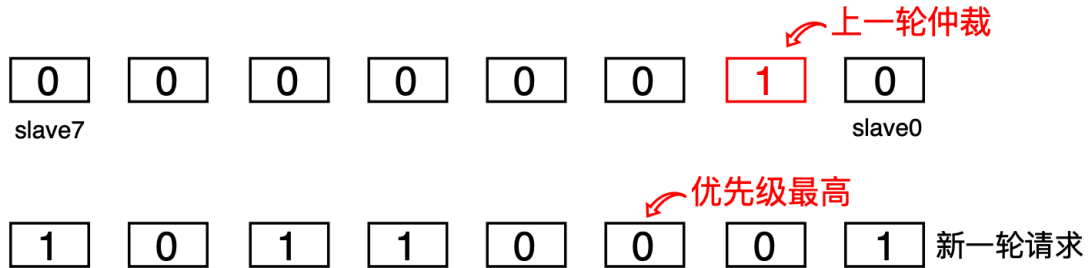
输入就是一堆请求，假设现在有8个slave，那输入就是8bit的data，相应bit位为1表示当前slave有请求。输出对应也是8bit，但是只能有1bit是1，因为仲裁只有1个，那也就是one-hot码。

我们假设 当前的输出(one-hot码)是0000_0010，只从这个我们就可以得到两个信息！

- 1、就是当前所仲裁的slave是1(总共是slave7-slave0)；
- 2、下一次仲裁的时候slave2的优先级是最高的，并且从右往左优先级依次降低，直到最高位后回到第0位绕回来，优先级依次降低。

我们要怎么 根据一个8bit的输入请求 来产生仲裁输出呢？

因为上一轮的仲裁输出grant[7:0]是0000_0010，那假设新一轮的请求req[7:0]是1011_0001，



因为上一轮仲裁的是slave1，那我们就需要从slave1往左找(因为离slave1的左边越近优先级越高)，找到第一个1，那这个“1”所在的slave就是这一轮仲裁的结果。如上图所示，这一轮的仲裁结果就是slave4。

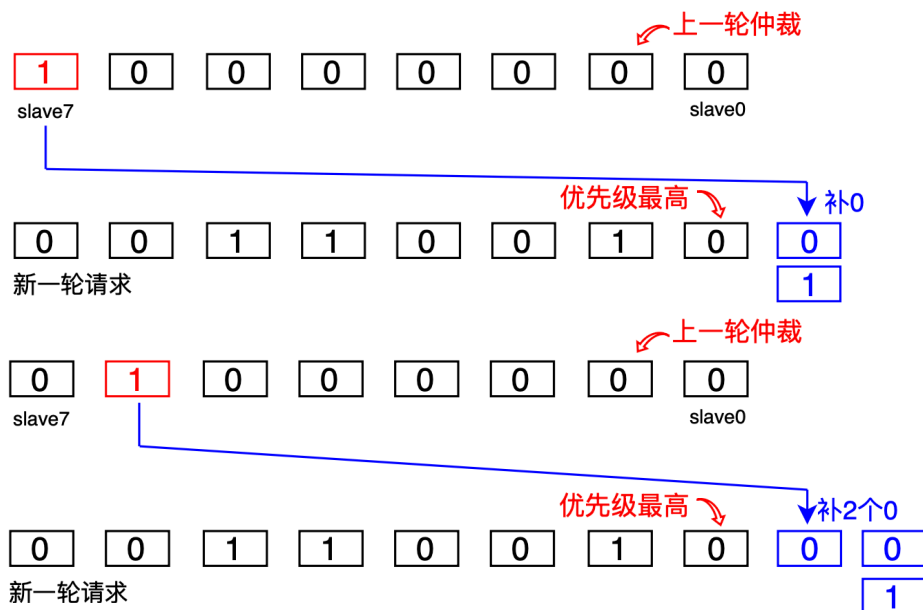
⚠️ 所以轮询仲裁器的编码重点就是 从当前的8bit请求中，找到离上一轮仲裁结果左边最近的那个“1”。这个与上面 固定优先级的仲裁器one-hot写法类似。

先说结果吧!!! (可以代入验证下，req=1011_0001，grant=0000_0010)

$$\text{grant_new} = \text{req}[7:0] \& \sim(\text{req}[7:0] - \text{grant}[7:0])$$

因为固定优先级req[0]优先级最高，因此是减去 1'b1，但是轮询仲裁器优先级不固定，因此就需要把1'b1移到优先级最高的位置，这正好就是上一轮的仲裁结果。

但是如果req[7:0]小于grant[7:0]的话，就不对了。需要先对req进行左移之后再减去grant。



左移几位要看grant所在的位置，如上图：grant[7]是1，就左移1位；grant[6]是1，就左移2位；grant[5]是1，就左移3位，以此类推(可以代入验证下，req=0011_0010，grant=1000_0000)。

那为了解决“不够减”的问题，现在的处理方法是把req信号 扩展成 $\text{double_req} = \{\text{req}, \text{req}\}$ 信号。
即： $\text{grant_new} = \text{double_req} \ \& \ \sim(\text{double_req} - \text{grant_old})$ ，因此轮询仲裁器比固定优先级仲裁器就多了这一步。

⚠ 这里需要注意下，轮询仲裁器需要保存上一级的仲裁结果，因此需要一拍才能出结果。

讲完这两个仲裁器，🤔 思考下这几个问题：

1、优缺点是什么？

- 固定优先级仲裁的优缺点
 - 优点： 面积小，关键路径延时小。
 - 缺点： 带宽利用率低，高优先级设备可能垄断总线，无法充分利用系统带宽，低优先级设备响应时间不可控。
- 轮询仲裁的优缺点
 - 优点： 公平性好，无饥饿问题。在请求负载均匀时，性能表现稳定。
 - 缺点： 仲裁逻辑比固定优先级稍复杂。如果只有一个高优先级设备频繁请求，它的延迟会比固定优先级方式大。

2、适用场景是什么？

- 固定优先级适合的场景：
 - 安全关键系统 – 保证高优先级任务始终优先
 - 实时控制系统 – 确定性响应时间要求
 - 简单嵌入式系统 – 资源受限，设计简单为主
 - 主从设备明确 – 主设备优先级明显高于从设备
- 轮询仲裁器适合的场景：
 - 多处理器系统 – 需要公平的资源分配
 - 网络交换机 – 需要公平的带宽分配
 - 负载均衡系统 – 需要动态调整优先级
 - 服务质量要求高 – 需要保证所有设备的服务质量

⚠ 仲裁器进阶(简略描述)：

1. 如何降低仲裁延迟？

- 可以将授权路径分为两级流水线，一级用于生成 pre_grant ，另一级用于更新指针和输出最终 grant 。

2. 如何减少面积？

- 上述掩码法需要双倍宽度的向量和桶式移位器，面积可能较大。
- 可以使用One-hot编码的指针，并用简单的选择器和优先级编码器来实现，面积可能更优。

3. 如何处理“锁”信号？

- 在实际总线（如AXI）中，主设备获得授权后，可能会通过一个 lock 信号表明它要连续进行多次传输，在此期间仲裁器不应改变授权。
- 设计时需要添加一个 locked 状态，当 lock 有效时，指针保持不变，直到传输结束。

4. 权重轮询仲裁器？

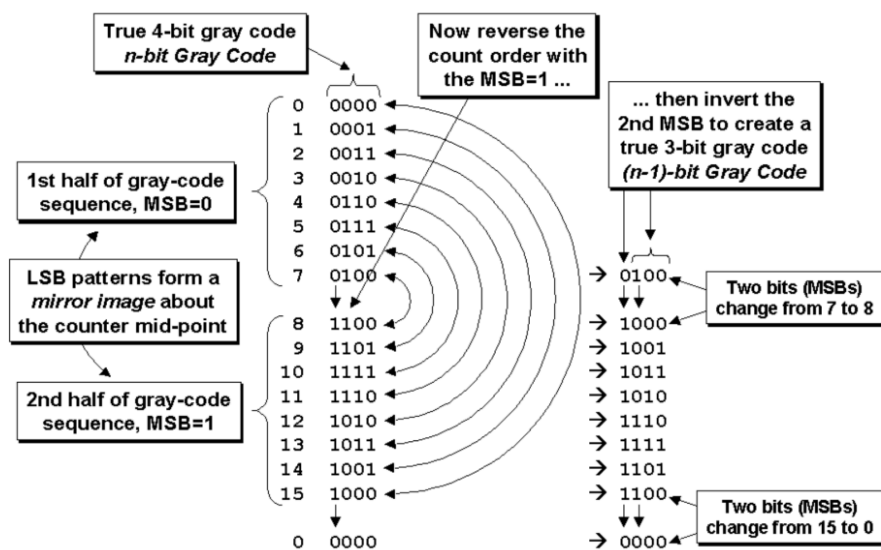
- 如果不同主设备的重要性不同，可以为每个设备分配一个权重（如信用值）。只有当设备的信用值消耗完后，指针才移动到下一个设备。这结合了公平性和差异性。

! 关于仲裁器常见面试问题总结

- 直接问题：
 - “请画出轮询仲裁器的框图。”
 - “请写出N路轮询仲裁器的RTL代码。”
 - “轮询仲裁器如何保证公平性？”
- 对比问题：
 - “轮询仲裁和固定优先级仲裁各在什么场景下更适用？”
- 深入问题：
 - “你实现的仲裁器是同步的还是异步的？为什么？”
 - “如果请求信号是异步的，该如何处理？”（提示：需要同步器）
 - “如何优化你设计的仲裁器的时序/面积？”
 - “如果有一个主设备需要被优先服务，但又不想完全用固定优先级，该如何设计？”（提示：权重轮询）
- 场景问题：
 - “在一个SoC中，CPU、DMA和GPU都要访问DDR，你会为它们选择哪种仲裁策略？为什么？”

格雷码与二进制转换

格雷码，也叫循环码或反射码，是一种二进制数码系统。它的核心特点是：**相邻的两个码组之间仅有一位不同**。这种特性使得它在数字电路（如异步同步，计数器、状态机）和位置编码器（如绝对式旋转编码器）中非常有用，可以有效地避免在状态变化过程中出现瞬时错误码。



1. 二进制码 → 格雷码

- 格雷码的最高位（最左边）等于二进制码的最高位。
- 格雷码的其余每一位，等于其对应的二进制位与它左边相邻的二进制位进行“异或（XOR）”操作的结果。
- 简而言之就是，将二进制码与逻辑右移的二进制码进行异或可得到格雷码。

//二进制数逻辑右移与自身进行异或逻辑运算

```
assign gray = (bin >> 1) ^ bin;
```

如果写到这就结束了，那这不是芯日记的风格!!!

🤔思考下，为什么左移 异或 得到的格雷码就只有1bit变化呢???

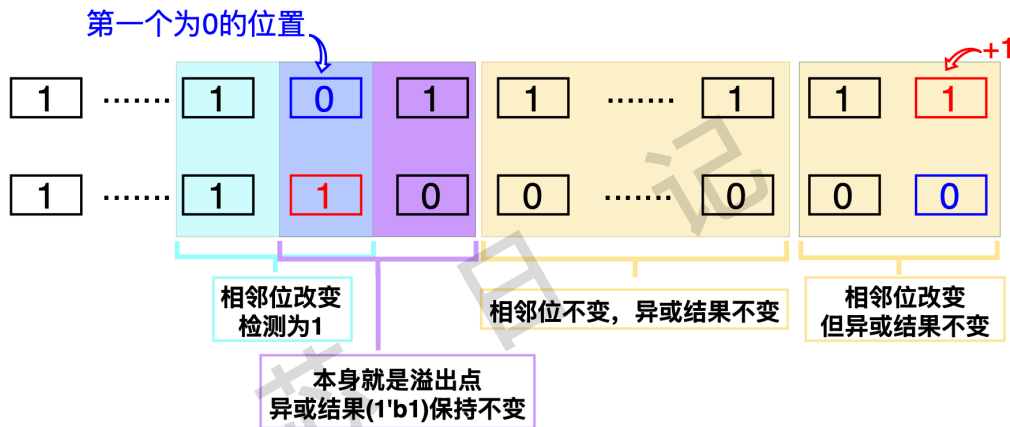
- 因为异或记录的不是数字本身，而是数字变化的“边界”。它告诉电路：“注意啦，从这里开始，状态要发生变化了！”由于每次只关注一个变化边界，所以自然就能保证每次只改变一位。

好像不够直观，举个例子好了。为什么一个二进制数加1会导致多bit位发生翻转呢？

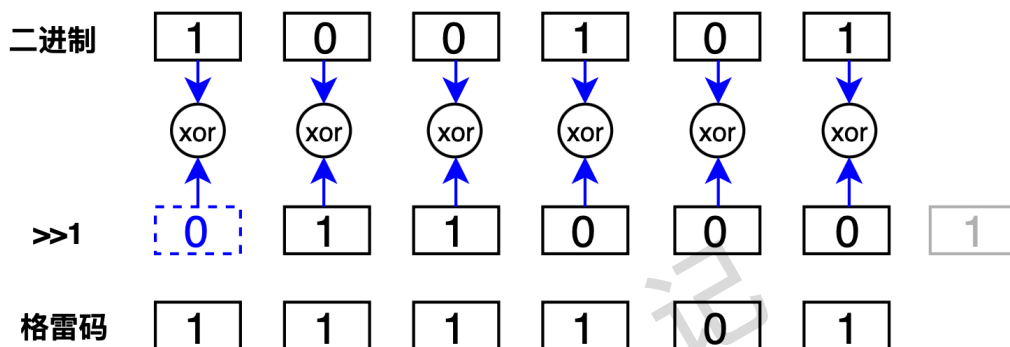
首先，如果这个二进制数data是偶数，也即data[0]是0，那么加1后，只有data[0]会发生翻转，其余位均不翻转，这是好事呀，要的就是只有1bit翻转。但是!! 如果data是奇数，也即data[0]是1，那么加1就会向上溢出，并且data[0]会翻转成0。

这个很好理解，假设data=3'b001，data[0]是1，加1后，data[0]会翻转成0，并且溢出到data[1]，使得data[1]也翻转。那问题来了，如果data是奇数，那么加1后，向上溢出到哪里呢?? 答案是会溢出到data[0]左边第一个为“0”的位置。

假设data=8'b1101_0111，data[0]是1，加1后，data[0]会翻转成0，并且向上溢出，但看到data[1]是“1”，因此会接着溢出，直到溢出到data[3]为“0”的位置结束。data[7:4]不受影响。



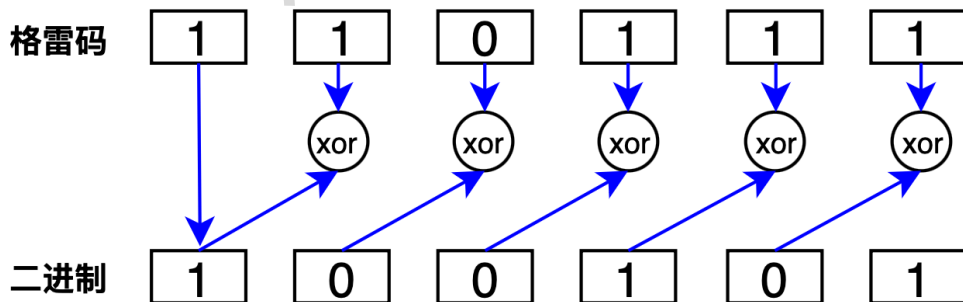
如上图所示，黄色矩形内虽然由“1”全变为了“0”，但是对异或来说没有影响。紫色矩形内本身就是溢出点，虽发生变化，但异或结果保持不变，只有蓝色框内的异或结果发生了变化。这就是左移异或的特性，对于+1这个操作，只有1bit翻转。



2. 格雷码 → 二进制码

- 二进制码的最高位（最左边）等于格雷码的最高位。
- 二进制码的其余每一位，等于其对应的格雷码位与已经计算出来的、它左边相邻的二进制位进行“异或（XOR）”操作的结果。

```
//verilog code as follow
generate
for(i=0;i<(N-1);i=i+1)
begin
assign binary[i] = binary[i+1]^gray[i];
end
endgenerate
assign binary[N-1] = gray[N-1];
endmodule
```



区别：

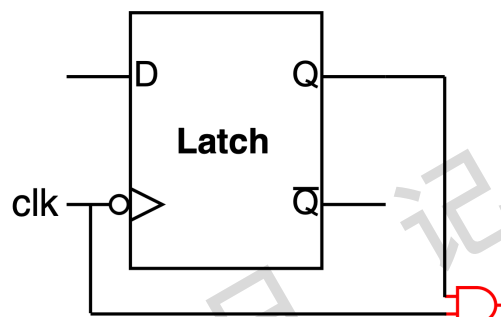
- 二进制转格雷码时，我们使用原始的二进制位进行计算。
- 格雷码转二进制码时，我们是一个递推的过程，依赖于上一步计算出的二进制位。

门控时钟(如何唤醒)

时钟门控是一种通过动态关闭时钟信号，来禁止寄存器不必要的翻转活动，从而降低电路动态功耗的技术。【关于功耗后续会单独详述：静态功耗/动态功耗】

核心思想：如果一个电路模块在某个时间段内不需要工作，那么就关闭它的时钟。时钟停止翻转，基于该时钟的寄存器就不会进行采样和更新，其后续的组合逻辑也就没有信号变化，从而大大减少了动态功耗。但简单在时钟路径上放置一个“与”或者“或”门，将会产生毛刺，这些毛刺会导致逻辑错误，甚至能摧毁整个芯片。

目前成熟的解决方案是：基于锁存器的时钟门控单元



```
always @(*) begin //生成latch
if (!clk_in)
en_latch = en;
end
```

```
assign clk_gated = en_latch & clk_in;
```

门控时钟的代码如上。

🤔思考下，enable信号怎么来呢？什么时候enable？什么时候disable呢？这个才是时钟门控的关键！电路谁不会画啊(谁直线不会加油啊)？

其实实际项目中，在RTL阶段并不会这么简单的写一个基于Latch的时钟门控。

一般会有两个区别！！

- 1、为了保证较高的生产缺陷覆盖率，会在clock gating中加入scan信号（即scan信号和使能信号en在进入锁存器前进行“或”），以保证在进行scan测试时，芯片中的所有触发器，无论使能信号是多少都能被时钟驱动，进而使得扫描链按照正常方式移动扫描数据。
- 2、另一个是加入 wakeup信号，去唤醒时钟。

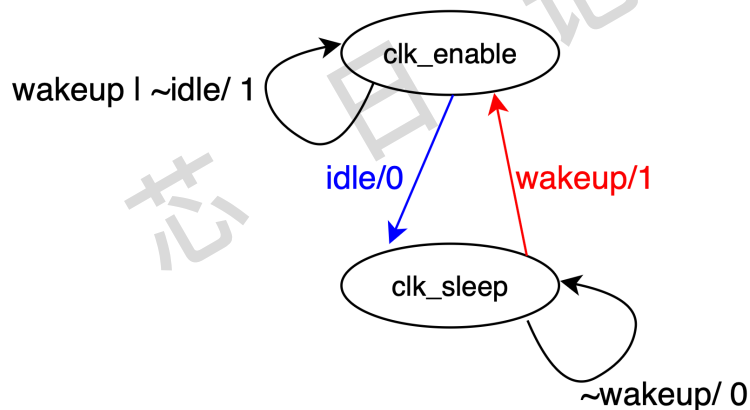
首先明确，什么时候要把时钟关掉？

答案是 module idle的时候，即内部pipeline没有valid，内部fifo为空，counter为0等。当module idle的时候，这个时候就不需要再引入时钟了，那就需要把时钟关掉以降低功耗！

什么时候唤醒时钟？

答案是 有输入的时候(wakeup=input_valid)，当有输入的时候要打开时钟接受数据。

因此就引入了用状态机写时钟门控，这也是HW面试官所问的问题，状态转移图如下图。



```
input  clk, enable_cg/*全局控制是否有时钟门控*/, idle, wakeup;
output clk_o;
```

```
always @* begin
    if(enable_cg) begin
        case(state)
            CLK_ENABLE: begin
                enableN = 1'b1;
                if(~WakeUp & idle) begin
                    enableN = 1'b0;
                    stateN = CLK_SLEEP;
                end
            end
            CLK_SLEEP: begin
                enableN = 1'b0;
            end
        endcase
    end
end
```

```

        if(WakeUp) begin
            enableN = 1'b1;
            stateN = CLK_ENABLE;
        end
    end
endcase
end else begin
    enableN = 1'b1;
    stateN = CLK_ENABLE;
end
end
end
register #(1) inst0(.out(state), .clk(clk), .in(stateN), .reset_(reset_));
register #(1,1'b1) inst1 (.out(enable), .clk(clk), .in(enableN), .reset_(reset_));
assign clk_gated = enable & clk_in;

```

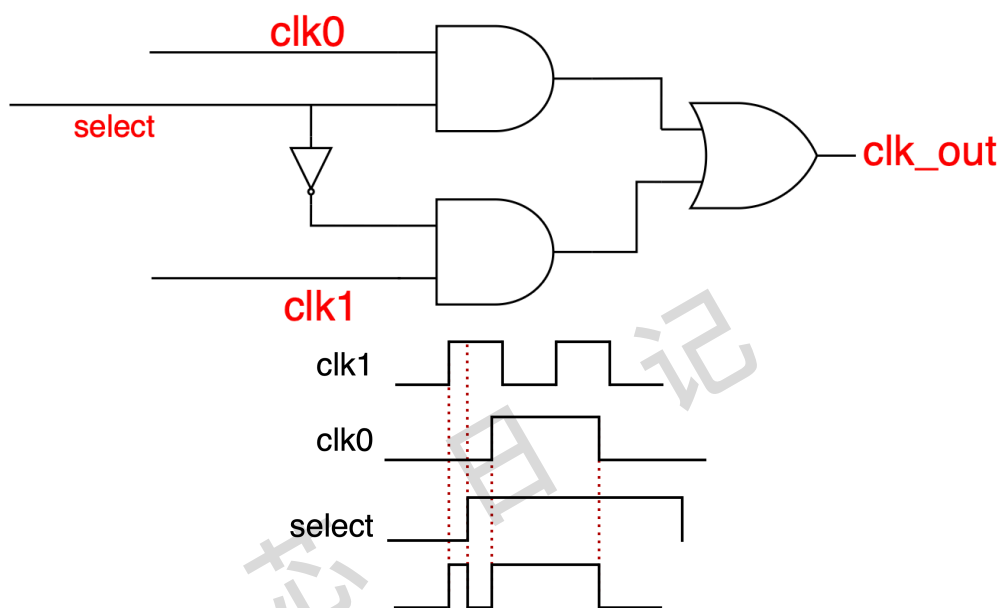
无毛刺时钟切换电路

时钟切换电路是指在一个数字系统中，动态地将一个模块的时钟源从一个时钟（例如 `clk\_a`）切换到另一个时钟（例如 `clk\_b`）。

为什么需要时钟切换

- 动态功耗管理（DVFS）：为了节省功耗，系统可能在性能要求高时使用高速时钟，在空闲时切换到低速时钟甚至关闭时钟。
- 功能模式切换：系统可能在不同工作模式下使用不同的时钟源，例如正常模式使用内部晶振，通信模式使用一个更精确的外部时钟。
- 时钟冗余与可靠性：在主时钟失效时，切换到备份时钟。

简单的时钟切换会产生毛刺

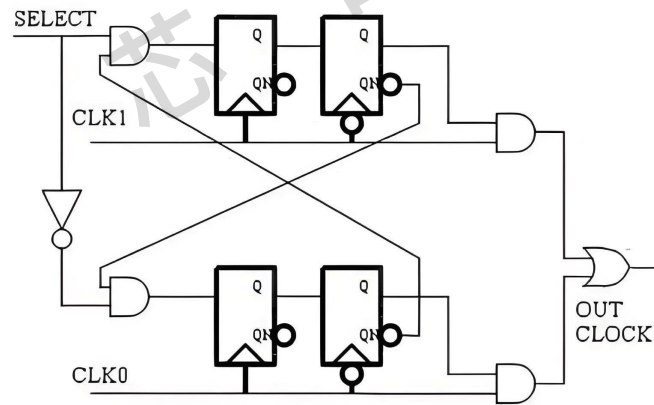


当选择信号 `select` 异步变化时，如果它发生变化的那一刻，`clk\_a` 和 `clk\_b` 正好处于不同的电平，就会产生一个非常狭窄的脉冲（即毛刺）。

无毛刺时钟切换的原理与电路实现

无毛刺切换的核心思想是：在目标时钟为低电平时，才允许改变选择信号的通路。这样可以确保切换只发生在时钟的“安全”区域（低电平期），从而避免在时钟有效边沿或高电平期间产生毛刺。

一个经典、可靠的实现方案如下：



//Verilog代码实现

```
always@(posedge clk0 or negedge rst_n)begin
    if (rst_n == 1'b0)
        clk0_f <= 1'b0;
    else
        clk0_f <= (~select) & (~clk1_ff);
end
always@(negedge clk0 or negedge rst_n)begin
    if (rst_n == 1'b0)
        clk0_ff <= 1'b0;
    else
        clk0_ff <= clk0_f;
end
```

```
always@(posedge clk1 or negedge rst_n)begin
    if (rst_n == 1'b0)
        clk1_f <= 1'b0;
    else
        clk1_f <= (select) & (~clk0_ff);
end
always@(negedge clk1 or negedge rst_n)begin
    if (rst_n == 1'b0)
        clk1_ff <= 1'b0;
    else
        clk1_ff <= clk1_f;
end
```



```
assign clk_out = (clk0_ff & clk0) | (clk1_ff & clk1);
```

串并转换

串并转换是指将串行数据与并行数据相互转换的过程。

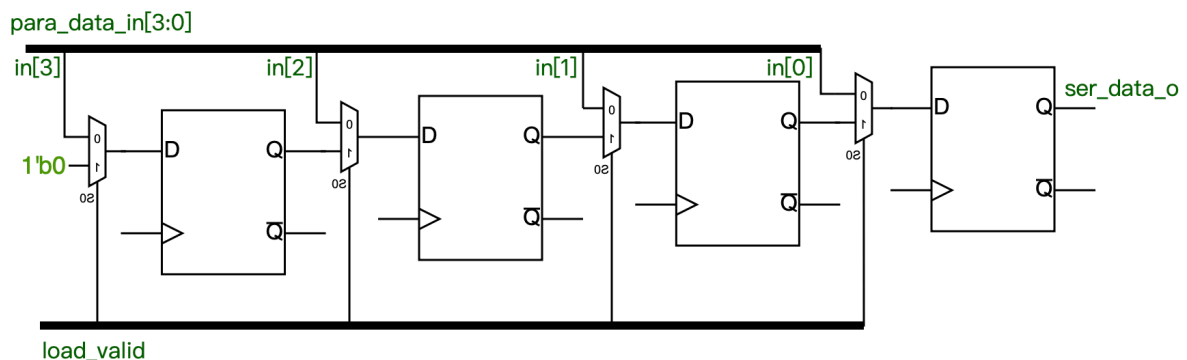
- 串行数据：数据位（0或1）在一根信号线上按时间顺序依次传输。
 - 特点：节省传输线，成本低，适合远距离通信，但速度相对较慢。
 - 例子：USB、PCIe、SATA、网络通信、UART（如RS-232）。
- 并行数据：数据的多个位通过多根信号线在同一时刻同时传输。
 - 特点：传输速度快，但需要多根线，成本高，容易产生信号间干扰，不适合远距离。
 - 例子：老式的打印机接口、CPU与内存之间的总线（如DDR）、单片机并口。

串行转并行 和 并行转串行 核心都是一个移位寄存器。

下面以8bit并转串为例：

```
always @(posedge clk or negedge rst_n) begin
    if (rst_n == 1'b0) begin
        ser_data_o <= 1'b0;
        data_buf <= 8'b0;
    end
    else if (load_valid == 1'b1)
        data_buf <= para_data_in;
    else
        data_buf <= data_buf << 1; //将寄存器内的值左移，依次读出
end
assign ser_data_o = data_buf[7];
```

手撕电路图如下(以4bit为例)



深度思考下：如果输入是持续来怎么办？如何知道已经接收/发送完了一组数据？

这里就需要引入计数器和FIFO，计数器的目的是记录当前数据移位情况。

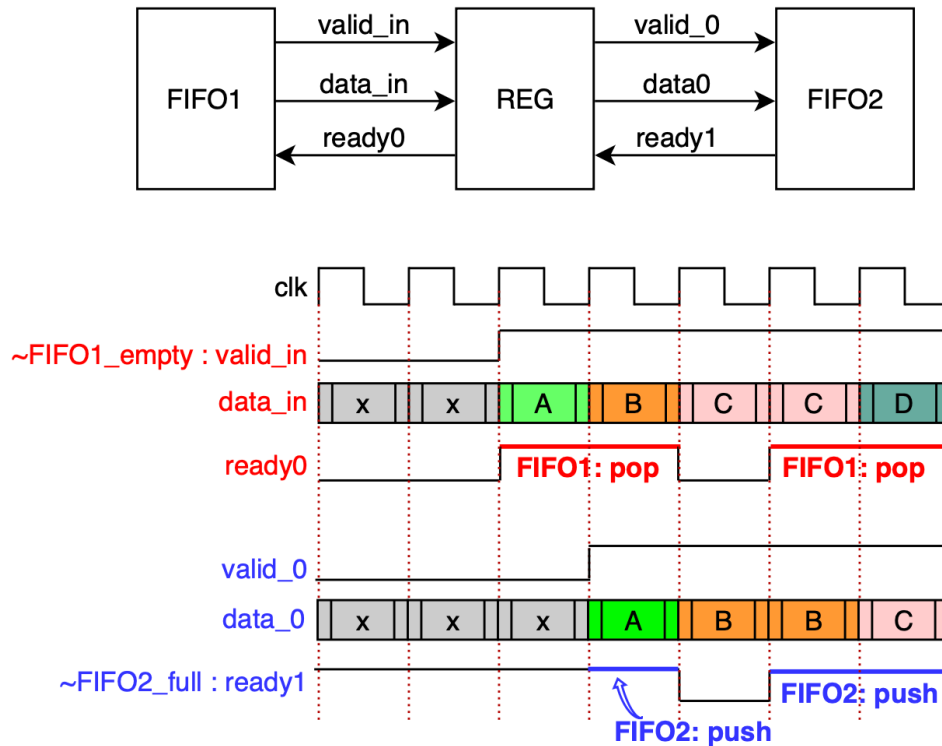
在串转并中：当计数器计满（例如从0计到7）时，产生一个数据有效 信号，通知外部电路可以读取并行数据了。同时，计数器可以复位，准备下一轮计数。

在并转串中：计数器用于控制“加载”信号的时机，并在计满后停止移位或开始下一轮的加载。

而FIFO是用来缓存输入data，FIFO宽度为输入位宽，而深度需要考虑输入时钟和输出时钟之间的关系(以并转串为例)，如果是同步时钟，并且输入N bit data持续输入，那么多深的fifo都会满。如果是异步FIFO，那FIFO的深度就取决于输入输出时钟频率和1个burst data的大小。

流水线握手、反压

流水线一般用于模块之间的握手。它确保数据只有在“发送方准备好发送”且“接收方准备好接收”时，才会被安全地传递。这是一种点对点、按需进行的同步机制。



Valid/Ready 握手协议详解，如上图所示：

握手一般有两根信号，valid和ready。

Valid：由发送方驱动。当`valid = 1`时，表示发送方放在数据总线上的数据是有效、可用的。

Ready：由接收方驱动。当`ready = 1`时，表示接收方已经准备好接收新的数据。

数据传输发生在时钟的上升沿，并且当`valid`和`ready`同时为高时完成。

现在分析上面那个典型场景：发送方是FIFO1，接收方是FIFO2。

1、当FIFO1有数据的时候，即非空(`~empty`)的时候表示发送方有数据有效，该有效信号(`valid_in`)和下游是否`ready`没有关系，如果下游`ready`，则当拍该数据(`data_in`)就可以被下游接受，也就是 `en_input = valid_in & ready0`；当下游接受到数据后，FIFO1的数据就可以被`pop`出来，如图上红色线。

2、如果下游一直`ready`，则数据当拍都可以被接受。假设FIFO2满了(`~full`)，那就需要向上反压，如图上`data_in = B`的时候，由于下游FIFO满了接受不了数据，那`valid_in`就需要再多持续一拍。直到FIFO2不满，则可以看到`data=C`持续了两拍。

// verilog

`valid_in = ~FIFO1_empty; // FIFO1不空，数据有效`

`ready0 = ~valid_in | ready1; // 下游ready，可以接收数据`

```

en_input = valid_in & ready0; // 上游数据有效，下游ready，完成握手，把数据打到下一拍，同时对FIFO1进行pop，处理上游的下一个数据
ready1 = ~FIFO2_full; // FIFO2满，不能接收，反压上游
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        valid0 <= 0;
    end else begin
        valid0 <= en_input? 1'b1 : ready0? 1'b0 : valid0; //当上游完成握手，下一级就可以拉高数据有效进行对其下游的握手；如果上游没有完成握手，且当前级的下一级ready，那么当前级的数据就要被消耗掉(valid0=1'b0)，否则valid0保持不变。
    end
end
FIFO2_push = valid0 & ready1; // 上游数据有效，当前FIFO2不满(即ready)，push到FIFO中

```

🤔 反压怎么理解

反压就是当接收方由于内部原因（如FIFO满、处理繁忙）无法处理更多数据时，通过反馈机制“反向”通知发送方暂停发送数据，从而防止数据丢失或系统拥塞。

- ready信号就是反压信号。
- 当接收方将ready拉低时，它实际上是在对发送方说：“我这边有压力了，请暂停发送！”
- 发送方在看到ready = 0时，必须保持当前的valid和data不变，直到ready重新变高。

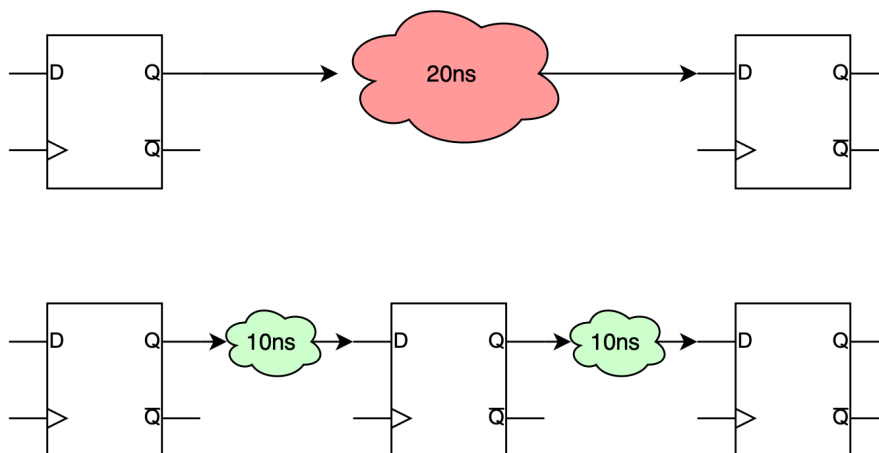
如上图FIFO2:

当FIFO未满时，它的ready1信号为高：“我可以接收数据”。

当FIFO满了时，它的ready1信号拉低：“我满了，别再送了！”

上游模块看到ready0为低，就不会再pop数据，保持当前数据不变，(data=C)，直到FIFO2被读出数据，有了空位，重新拉高ready1。

你可以这样理解：我们使用valid/ready握手协议来构建模块间的接口，而这个协议天然地赋予了系统反压的能力。当你在代码中写下 `if (ready) ...` 时，你就是在实现反压逻辑。



另外流水线也可以用来修timing问题，如上图。两级reg之间的握手就是上述的valid和ready信号。只有当上级valid和下级ready，上级才可以把数据传给下级，否则数据和valid都保持不变，等待下级ready。

🤔：流水线设计注意事项⚠️⚠️⚠️：

- 1、流水线最好划分在数据通路上位宽较小的地方，以节省寄存器数量和面积。
- 2、流水线每一级的关键路径延时最好接近，这样利于获得最大的timing margin，频率可以跑到最高。如上图均分为10ns，这样频率最高可以100MHz，但是如果不均分，最高频率肯定小于100M。

异步复位同步释放

🤔为什么需要这种复位方式？

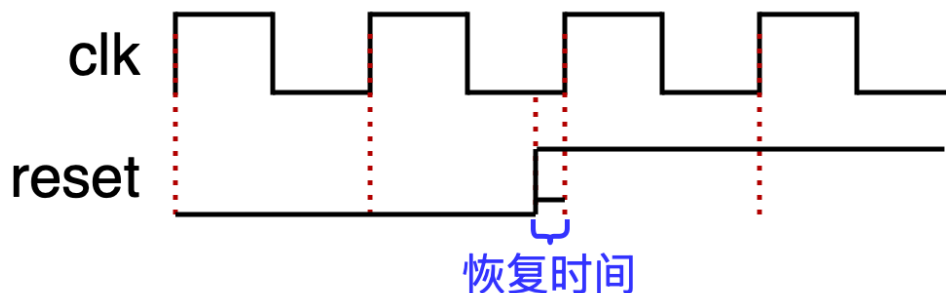
避免复位释放亚稳态，如果异步复位信号在时钟边沿附近撤销：

- 可能违反触发器的 恢复时间 和 移除时间；
- 导致输出亚稳态，系统进入不确定状态；
- 同步释放确保了复位撤销发生在确定的时钟边沿。

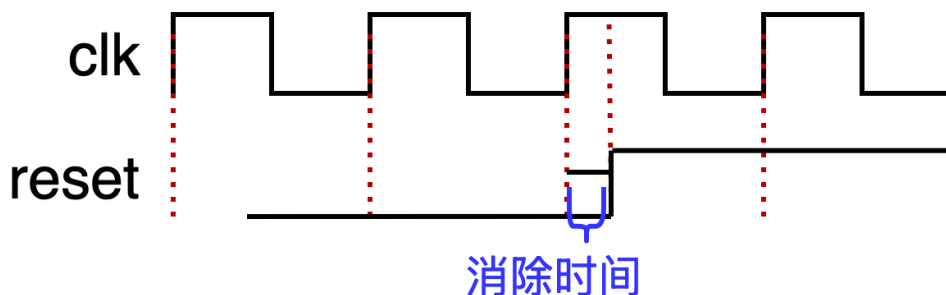
🤔为什么异步复位信号会有亚稳态？

异步复位的亚稳态问题不是出现在复位生效时，而是出现在复位撤销时。其根本原因是复位信号的撤销违反了触发器的建立时间和保持时间。

异步复位信号释放（对低电平有效的复位来说就是上跳沿）与紧跟其后的第一个时钟有效沿之间，有一个必须间隔的最小时间称为**恢复时间**(recovery time)。



时钟有效沿与紧跟其后的异步复位信号释放之间所必须的最小时间称为**消除时间**(removal time)。小于这个时间，则触发器的输出端的值将是不确定的，可能是高电平，可能是低电平，可能处于高低电平之间，也可能处于震荡状态，并且在未知的时刻会固定到高电平或低电平。这种状态就称为亚稳态。



1 同步复位原理：同步复位只有在时钟沿到来时复位信号才起作用，则复位信号持续的时间应该超过一个时钟周期才能保证系统复位。

2 异步复位原理：异步复位只要有复位信号系统马上复位，因此异步复位受毛刺影响较大，抗干扰能力差。

🌟 主要区别

是否有时钟信号参与。异步复位不需要时钟参与，一旦信号有效立即执行复位操作；同步信号需要时钟参与，只有有效的时钟信号出现，复位信号才有效。

🌟 优缺点

同步复位

优点 😊 1 有利于仿真器的仿真。

优点 😊 2 可以使所设计的系统成为100%的同步时序电路，这便大大有利于时序分析，而且综合出来的最大时钟频率一般较高。

优点 😊 3 因为他只有在时钟有效电平到来时才有效，所以可以滤除高于时钟频率的毛刺。

缺点 😞 1 复位信号的有效时长必须大于时钟周期，才能真正被系统识别并完成复位任务。同时还要考虑，诸如：clk skew，组合逻辑路径延时，复位延时等因素。

缺点 😞 2 由于大多数的逻辑器件的目标库内的DFF都只有异步复位端口，所以，倘若采用同步复位的话，综合器就会在寄存器的数据输入端口插入组合逻辑，这样就会耗费较多的逻辑资源。

🌟 优缺点

异步复位

优点 😊 1 大多数目标器件库的dff都有异步复位端口，因此采用异步复位可以节省资源。

优点 😊 2 设计相对简单。

优点 😊 3 异步复位信号识别方便，而且可以很方便的使用FPGA的全局复位端口GSR。

缺点 😞 1 在复位信号释放（release）的时候容易出现问題。具体就是说：倘若复位释放时恰恰在时钟有效沿附近，就很容易使寄存器输出亚稳态。

缺点 😞 2 复位信号容易受到毛刺的影响。

✅ 异步复位同步释放

指复位信号是异步有效的，即复位的发生与clk无关。后半句“同步释放”是指复位信号的撤除也与clk无关，但是复位信号是在下一个clk来到后起的作用（释放）。

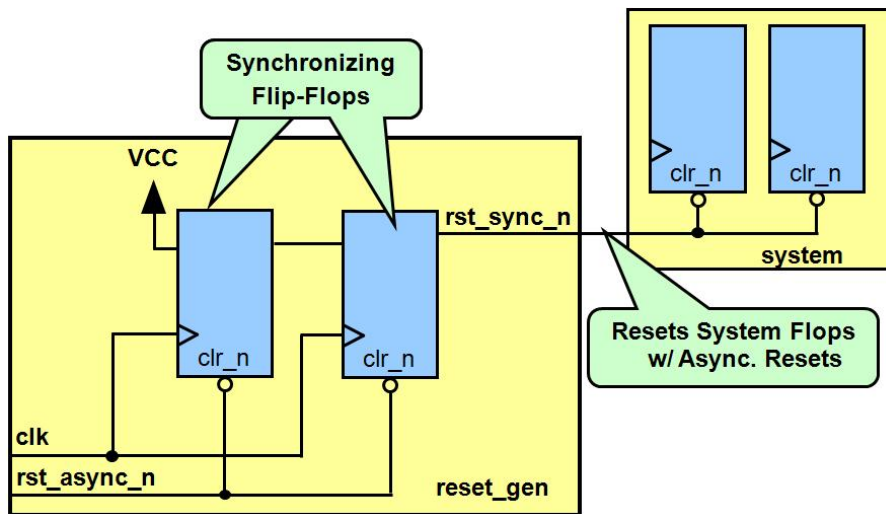
如下图第一个方框内是异步复位和同步释放电路。有两个D触发器构成。第一级D触发器的输入时VCC，第二级触发器输出是可以异步复位，同步释放后的复位信号。

```
module reset_gen(output rst_sync_n, input clk, input rst_async_n);
reg rst_d1, rst_d2;
always @ (posedge clk, posedge rst_async_n)
    if (rst_async_n)
        begin
            rst_d1 <= 1'b0;
            rst_d2 <= 1'b0;
        end
    else
        begin
            rst_d1 <= 1'b1;
        end
end
```

```

rst_d2 <= rst_d1;
end
wire rst_sync_n = rst_s2; //异步复位同步释放，输出的复位信号

```



模3检测

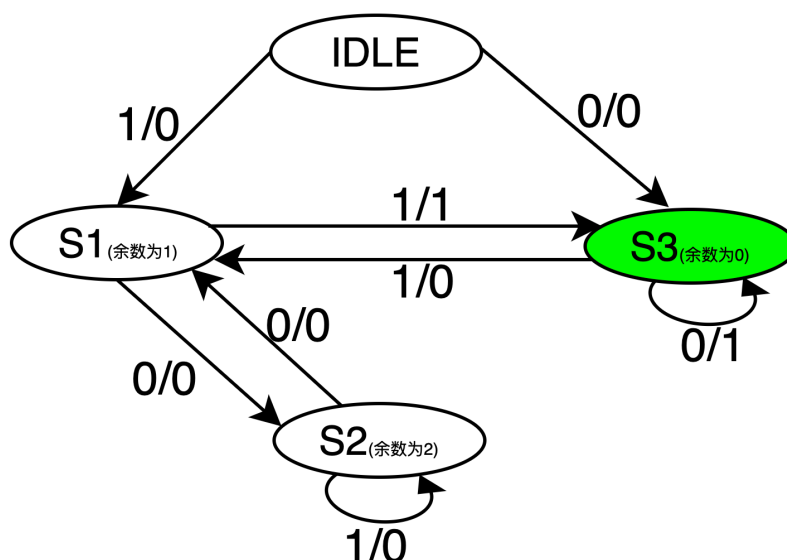
题目：使用verilog代码，设计电路，判断输入序列能否被三整除，能的时候输出1，不能的时候输出0。

当输入序列中存在模三余数为0的子序列时，检测器会输出一个逻辑“1”信号；否则，输出逻辑“0”信号。这种检测器可以应用于**数字通信、计算机网络等领域**，用于实现数据传输和错误检测等功能。模三检测器的实质是一个判断输入序列的序列检测器

🤔原理分析：一个输入被三除，对于余数来说只存在三种可能，分别为0，1，2（被整除，余数为1，余数为2），那么假如0，1，2对应于三种状态，加上IDLE的默认状态，我们需要四个状态，来表示模三检测器的状态机。

由于输入序列是一边移位，一边输入，即假如第一位输入1，第二位输入0，在输入第二位的时候，当前输入序列表示为10，即先输入的1，移动到了输入序列的高位处了。左移就相当于乘2。

状态转移图如下：



状态转移表如下：

当前态余数	输入	次态余数	输出
0	1	1	0
0	0	0	1
1	0	2	0
1	1	0	1
2	0	1	0
2	1	2	0

```
parameter IDLE =2'b00,
s1 =2'b01, // 余数为1
s2 =2'b10, // 余数为1
s3 = 2'b11; // 余数为0
reg [1:0] state,state_nxt;
always@(posedge clk or negedge rst_n)
begin
if(!rst_n)
state<=IDLE;
else
state<=state_nxt;
end

always@(*)
begin
case(state)
IDLE: nstate = data?s1:s3;
s1: nstate = data?s3:s2;
s2: nstate = data?s2:s1;
s3: nstate = data?s1:s3;
default nstate = IDLE;
endcase
end
assign test = state==s3?1:0;
```

异步电路多bit同步–握手

参见异步电路CDC内容！

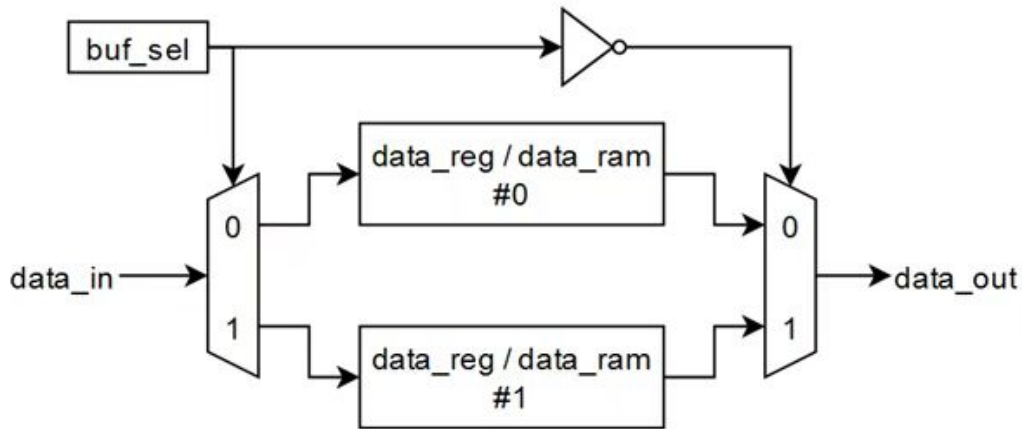
乒乓操作

乒乓操作主要是为了解决了数据处理速度不匹配和保证数据流连续性的核心问题。

1、乒乓操作的核心概念

乒乓操作的核心是利用两块缓冲区，通过切换数据通路，让数据“写入”和“读出”在两个缓冲区之间交替进行，从而实现：

- 写操作和读操作在时间上完全并行。
- 从整体上看，数据流无间断、无阻塞地连续处理。



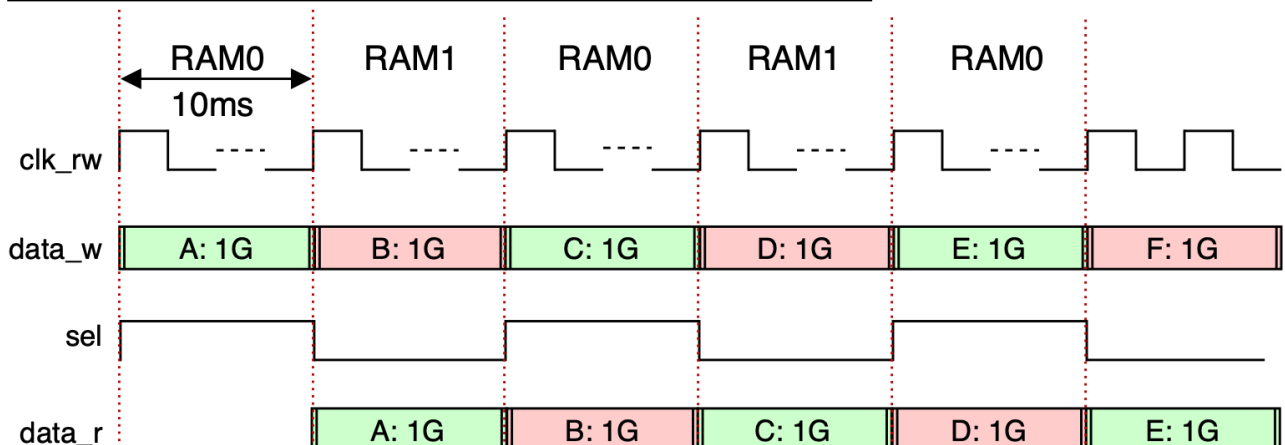
2、基本结构

- 两个存储单元 (Buffer A & B)：通常是RAM、FIFO或一组寄存器。
- 一个多路选择器 (MUX) 网络：控制数据源的切换。
- 一个控制逻辑 (FSM)：产生缓冲区选择、读写使能等控制信号。
- 两个数据通路：
 - 输入数据流 -> (通过MUX选择) -> 写入当前非工作缓冲区。
 - 读取当前工作缓冲区 -> (进行后续处理) -> 输出数据流。

3、工作流程（像打乒乓球一样来回切换）：

- 第一阶段：输入数据流写入 Buffer A。
- 第二阶段：输入数据流切换至写入 Buffer B；同时，处理模块切换至从 Buffer A 中读取数据。
- 第三阶段通过输入数据选择单元的再次切换将输入的数据流缓存到BufferA，同时将Buffer B缓存的第二个周期数据通过输出数据流选择单元切换送到数据流运算处理模块，进行运算处理。
- 如此循环往复，像打乒乓球一样，A和B的角色（“写缓冲区”/“读缓冲区”）定期交换。

假设输入数据流的速率为100Mbps，乒乓操作的缓冲周期是10ms。



第一个缓冲周期(10ms)，将输入数据流缓存到“RAM0”。

第二个缓冲周期(10ms)，通过“输入数据选择单元”的切换，将输入数据流缓存到“RAM1”，同时将“RAM0”缓存的第一个周期的数据通过输出数据流选择单元的选择送到数据流运算处理模块进行运算处理。

第三个缓冲周期，通过输入数据选择单元的再次切换将输入的数据流缓存到“RAM0”，同时将“RAM1”缓存的第二个周期数据通过输出数据流选择单元切换到数据流运算处理模块，进行运算处理。如此循环。

乒乓操作的最大特点是通过“**输入数据选择单元**”和“**输出数据选择单元**”按节拍、相互配合的切换，将经过缓冲的数据流没有停顿地送到“**数据流运算处理模块**”进行运算与处理。

思考下🤔？这个直接用FIFO有啥区别？这不就是打拍吗？

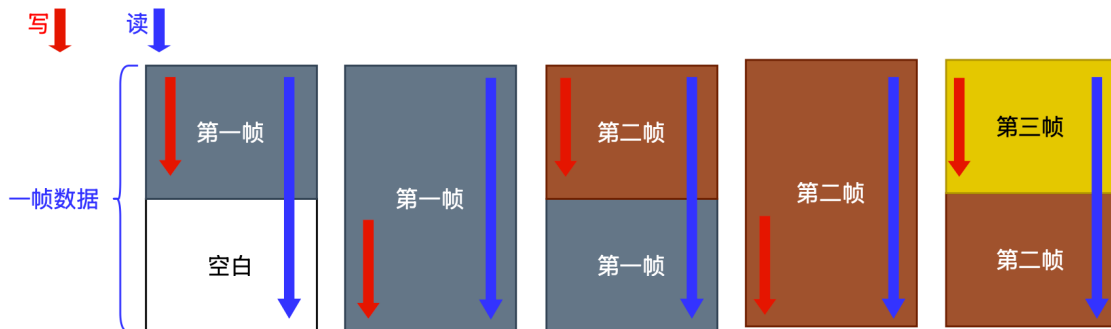
这个要结合乒乓buffer的应用场景来看，乒乓buffer注重的数据的完整性，一般常用于图像处理，因为图像处理需要对整帧的数据进行处理，在处理整帧数据的同时要保持数据不变。然而整帧的数据量较大，需要将每帧的数据逐行去写入到存储器。因此如果对每行数据进行打拍的话，势必会影响数据的完整性。

结合下面这个场景会好理解一些！

图像处理经常会用DRAM来作为中间缓存。先假设图像生成的帧率是30fps，图像处理输出的帧率是60fps。假设一个图像的分辨率是 $3 \times 2 = 6$ ，数据为：1, 2, 3, 4, 5, 6。生成一个像素的时间假设是2秒，处理一个像素的时间是1s（因为帧率是2倍的关系）。

在6s的时间里，处理模块就需要请求了一帧数据，可这个时间里只生成了1, 2, 3共三个像素，那处理模块就需要等待。在12s的时间里，处理模块需要请求两帧数据，可这个时间里只能生成一帧数据。因此如果中间没有缓存，输出模块就总是要不全一帧完整的图像。

假设中间加入缓存，但没有乒乓操作。如下图，红线为写，黑线为读。



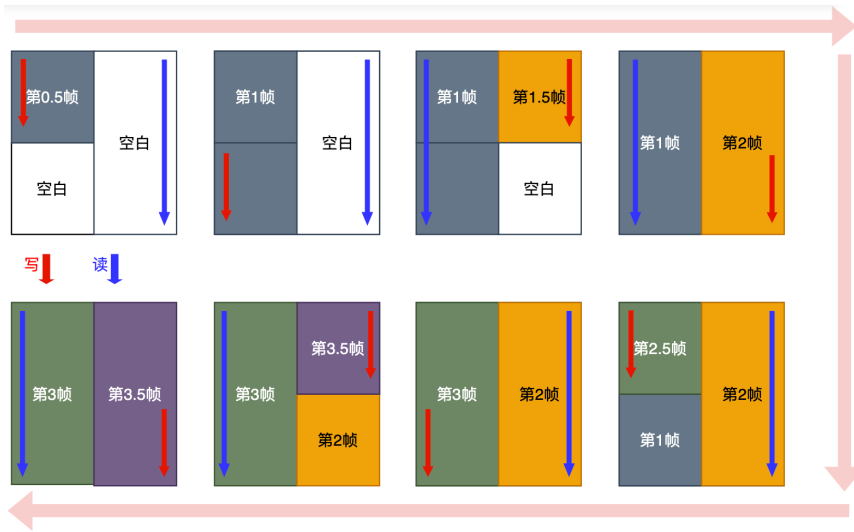
由于一帧图像实际情况是一行行的生成和读取的，由于图像生成和处理的帧率分别是30fps和60fps，仿佛刚好可以写一帧读两帧。

但是一帧数据并不是瞬间生成的，所以会出现处理模块从缓存中读的上半帧是新帧，而由于缓存的下半帧还没有被写完（图像生成的帧率较慢），因此处理模块读的下半帧还是老帧。示意图如上。将上述5帧图像生成的时间点编号为 t_1 、 t_2 、 t_3 、 t_4 、 t_5 。在 t_1 、 t_3 、 t_5 时刻图像都是残缺帧（新老帧参半），在 t_2 、 t_4 时刻图像才是完整的一帧，但是处理模块每个时间点都需要完整的一帧图像，这就是错帧现象。而解决错帧现象的方法则是**乒乓操作**。

乒乓操作，使用两个缓存区，写缓存区1时读缓存区2，写缓存区2时读缓存区1，读写交替。如下图，红线为写，黑线为读。从图中可以看出，乒乓操作中，每次读的（黑线）都是完整的帧，每一帧读2次，这样便没有出现读残缺帧的现象，解决了错帧问题。

把乒乓操作模块当做一个整体，站在这个模块的两端看数据，输入数据流和输出数据流都是连续不断的，没有任何停顿，因此非常适合对数据流进行流水线式处理。所以乒乓操作常常应用于流水线式算法，完成数据的无缝缓冲与处理。

通过乒乓操作实现低速模块处理高速数据的实质是：通过DPRAM 这种缓存单元实现了数据流的串并转换，并行用“数据预处理模块0”和“数据预处理模块1”处理分流的数据，用面积换速度！



在上述设计中，由于读写速率为1:2，所以写一帧读两帧即可，但是很多时候读写速率的比例是其他数值，那怎么办呢？

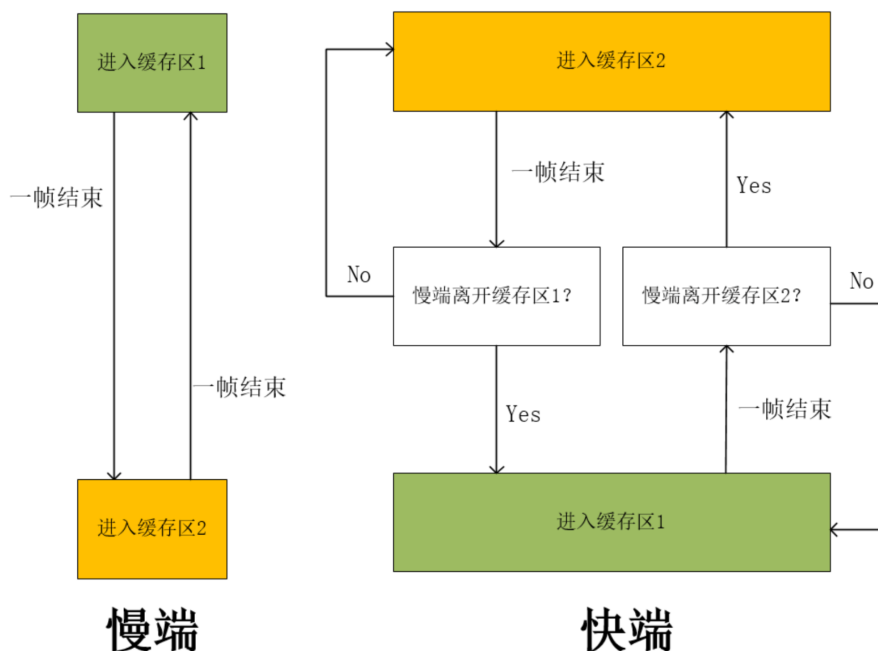
(1) 写端在缓存区1写完一帧数据就切换到缓存区2写下一帧，写完后又切换回缓存区1写再下一帧，如此反复。

(2) 读端在缓存区2读完一帧：

①如果写端仍然在缓存区1，则读端不切换缓存区，而是继续在缓存区2重读一帧。

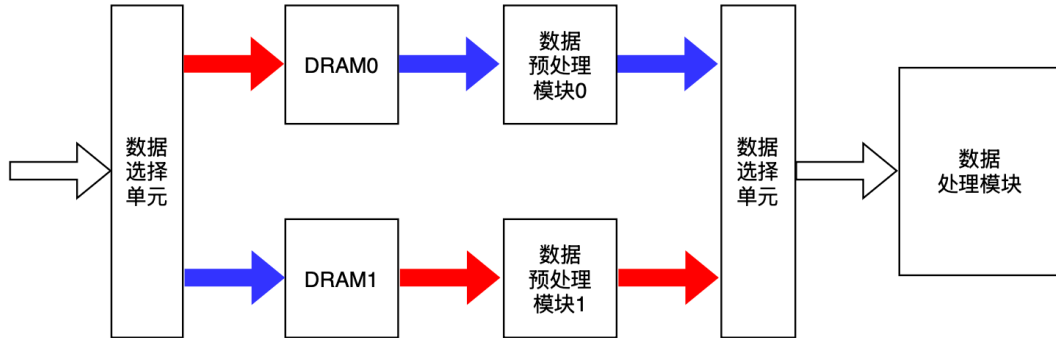
②如果写端离开了缓存区1，即切换到缓存区2了，说明写端已经在缓存区1写完了，则读端可以切换到缓存区1。这样就能保证读端每次读的都是完整的帧，就算有一小段时间是读写端都在同一个缓存区，但是由于写慢读快，写是不会追上读的，读还是能读完旧帧，之后旧帧才被新帧覆盖。

写慢读快这样设计就行，那如果写快读慢呢？其实也是一样的思想，反过来即可。无论谁快谁慢，都是快端照顾慢端，即慢端一帧结束就离开自身缓存区到另一缓存区。快端则一帧结束后，先

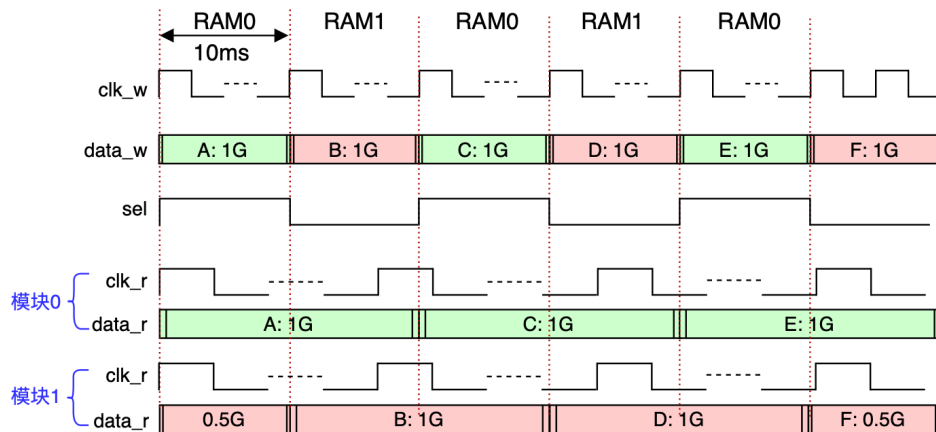


检测慢端是否离开自身缓存区，是则切换过去，否则不切换过去，而是重新在自身缓存区再工作一帧。示意图如上图所示。

!!! 乒乓操作还可以达到用**低速模块处理高速数据流**的效果。如下图所示，数据缓冲模块采用了双口RAM，并在DPRAM后引入了一级数据预处理模块，这个数据预处理可以根据需要的各种数据运算，比如在WCDMA设计中，对输入数据流的解扩、解扰、去旋转等。



波形示意图如下。



!!! 乒乓操作的典型应用场景

乒乓操作的本质是用空间（两块存储区）换时间（无停顿流水线）和解决时钟域/速度匹配问题。主要应用在：

1. 跨时钟域数据缓冲：

- 场景：从ADC采样的高速数据（CLK_A）需要传给一个处理速度较慢的算法模块（CLK_B）。
- 应用：在时钟域A和B之间插入一个基于异步FIFO或双口RAM的乒乓缓冲区。一个缓冲区分在CLK_A下被写满后，切换给CLK_B读出，同时另一个缓冲区分在CLK_A下接收新数据。这有效隔离了时钟域，避免了亚稳态扩散，并平滑了速率差。

2. 数据流无损处理：

- 场景：对连续数据流（如视频像素流、网络报文）进行需要固定处理时间的操作（如FIR滤波、图像卷积、加密解密）。
- 应用：处理模块处理Buffer B的数据需要N个周期。在这N个周期内，新的输入数据可以毫无阻碍地写入Buffer A。当处理完成，立刻切换读写目标，处理模块开始处理Buffer A，而新数据写入Buffer B。从宏观看，数据处理是连续的，没有“气泡”。

3. 数据包/帧的拼接与拆分：

– 场景：接口数据位宽与处理核心位宽不匹配。例如，接收到的是32位宽的数据流，但内部DSP核需要一次处理128位数据。

– 应用：使用乒乓缓冲区作为位宽转换器。Buffer A收集4个32位数据拼成1个128位数据包，同时Buffer B将上一个拼好的128位数据提供给DSP。完成后角色互换。

4. 异步接口数据预取：

– 场景：CPU通过APB/AXI总线从低速外设读取数据，但CPU不希望因等待数据而停顿。

– 应用：DMA或专用控制器使用乒乓操作预取数据。当CPU读取Buffer A的数据时，DMA正在将下一批数据写入Buffer B。这样CPU总能从一块已准备好的缓冲区中快速读取数据，感知不到外设的延迟。

! 面试题目与解答思路

题目1：请解释什么是乒乓操作（Ping-Pong Operation），它解决了什么问题？

– 考察点：基本概念理解。

– 回答思路：

1. 定义：一种利用两个存储单元交替进行读写，以实现数据流无间断处理的技术。

2. 核心：空间换时间，读写并行。

3. 解决问题：

– 速率匹配：解决生产者和消费者速度不一致的问题。

– 流水线停顿：避免处理单元因等待数据而空闲，提高系统吞吐率。

– 数据连续性：保证像视频、音频等流式数据的处理不间断。

题目2：如果让你用Verilog设计一个简单的乒乓缓冲区，输入输出数据位宽均为8位，深度为256，你会如何设计？描述主要模块和状态机。

– 考察点：RTL设计能力，对状态机和控制逻辑的理解。

– 回答思路：

1. 顶层模块：声明两个双口RAM（或寄存器数组）作为Buffer A和B。

2. 控制FSM设计（关键）：

– 状态IDLE：初始状态，准备接收数据。

– 状态WRITE_A_READ_B：`sel = 0`，输入数据写入A，从B读取数据到输出。当A写满（或B读空）时，准备切换。

– 状态SWITCH：一个短暂的过渡状态，确保读写指针复位或切换信号稳定，避免冲突。

– 状态WRITE_B_READ_A：`sel = 1`，输入数据写入B，从A读取数据到输出。当B写满（或A读空）时，切换回状态2。

3. 辅助逻辑：为每个RAM生成独立的写使能、写地址、读使能、读地址。这些信号由FSM状态和`sel`信号控制。

4. 多路选择器：根据`sel`信号选择将输入数据导向哪个RAM，以及将哪个RAM的数据输出。

5. 注意事项：需要处理好缓冲区的“满”和“空”条件，防止覆盖未读数据或读空。面试时可以讨论是用计数器、标志位还是比较地址来实现。

题目3：乒乓操作和FIFO（先入先出队列）有什么区别和联系？在什么情况下你会选择乒乓操作而不是FIFO？

– 考察点：知识对比和工程权衡。

– 回答思路：

– 联系：两者都是数据缓冲机制，用于解耦生产者和消费者，平滑流量。

– 区别：

1. 结构：FIFO是单一块存储区，顺序读写；乒乓是两块存储区，整块切换。
2. 存取方式：FIFO是流式的，随来随走；乒乓是块式的，写满一块/读空一块才切换。
3. 控制复杂度：标准同步FIFO有成熟IP，控制简单（仅`full/empty`）；乒乓需要自定义状态机控制切换。

4. 延迟：乒乓操作的延迟是固定的（一块数据的处理时间）；FIFO的延迟可变。

– 选择乒乓的场景：

1. 后续处理模块必须以固定大小的数据块为单位进行操作（如128点FFT、一个图像宏块）。
2. 需要保证数据块的完整性和处理的确定性时延。
3. 当处理模块的启动开销较大，更适合整块数据处理时。
4. 当两块独立的SRAM资源可用，且块处理逻辑清晰时，乒乓的实现可能比深度很大的FIFO更高效。

题目4：分析一下乒乓操作的优缺点。

– 考察点：全面思考能力。

– 回答思路：

– 优点：

1. 高吞吐率：读写完全并行，理论上能实现100%的带宽利用率。
2. 无阻塞连续流：理想情况下，数据流不会中断。
3. 结构清晰：对于块处理算法非常友好，易于模块化设计。
4. 易于实现位宽/时钟域转换：结合双口RAM和异步FIFO思想很容易扩展。

– 缺点：

1. 资源开销：需要两倍的存储资源。
2. 固定延迟：至少引入一块数据深度的延迟，不适合极低延迟要求的场景。
3. 控制逻辑相对复杂：需要精心设计状态机来避免切换时的竞争和冒险。
4. 对数据流不规律的情况不友好：如果数据块大小不固定或间隔随机，切换条件难以确定，可能造成效率下降（此时FIFO更优）。

静态时序分析

异步电路CDC

异步时序设计指的是在设计中有两个或以上的时钟，且时钟之间是同频不同相或不同频率的关系。而异步时序设计的关键就是把数据或控制信号正确地进行跨时钟域传输。



⚠ 将异步电路之前，这里先详细讲下亚稳态：

每一个触发器都有其规定的建立(setup)和保持(hold)时间参数，在这个时间参数内，输入信号在时钟的上升沿是不允许发生变的。如果在信号的建立时间中对其进行采样，得到的结果将是不可预知的，即亚稳态。

触发器进入亚稳态的时间可以用参数 MTBF(mean time between failures)来描述，MTBF即触发器采样失败的时间间隔，其公式描述如下：[1]杜旭,王夏泉.ASIC中的异步时序设计[J].微电子学,2004,34(5): 522-524.

$$MTBF = e^{t_r/\tau} / T_0 * f * a$$

其中：

t_r = 分辨时间(时钟沿开始),

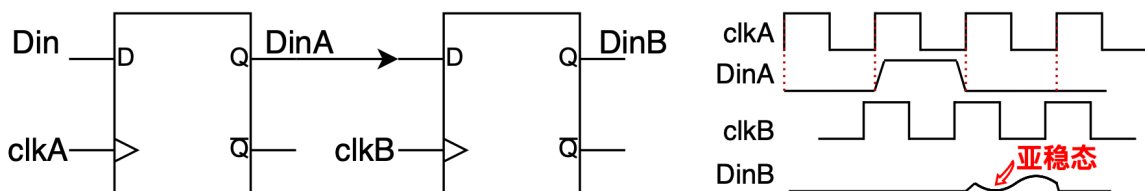
τ 、 T_0 = 触发器参数,

f = 采样时钟频率,

a = 异步事件触发频率。

MTBF越大说明系统采样失败的可能越小。对于高速(频率较高)的设计，MTBF越小，采样失败的概率越大。对于一个典型的0.25 μ m工艺的ASIC库中的一个触发器，取以下参数：

$t_r = 2.3\text{ns}$, $\tau = 0.31\text{ns}$, $T_0 = 9.6\text{as}$, $f = 100\text{M Hz}$, $a = 10\text{M Hz}$: $MTBF = 2.01\text{ days}$, 即触发器每两天便可能出现一次亚稳态。



两级同步器：

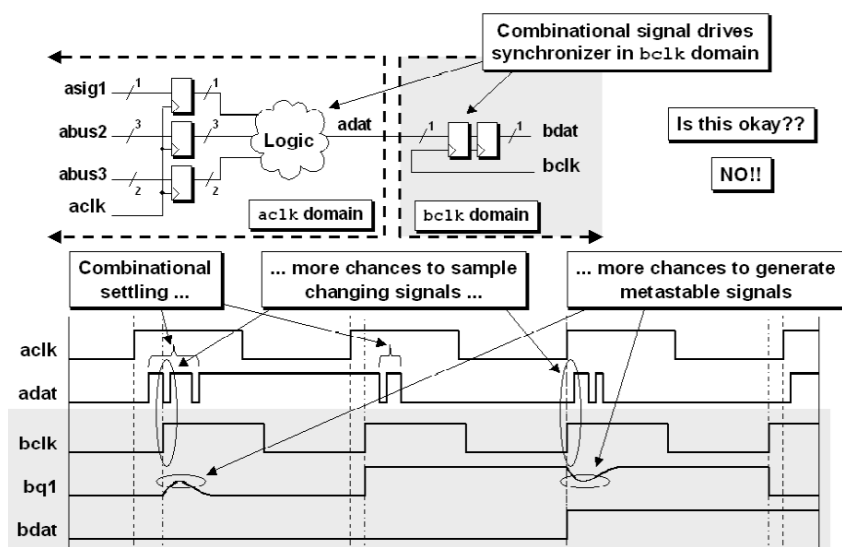
当使用了双锁存器以后，DinB的MTBF由以下公式可以得出：

$$MTBF = e^{t_r/\tau} / T_0 f_a * e^{t_r/\tau} / T_0 f_a$$

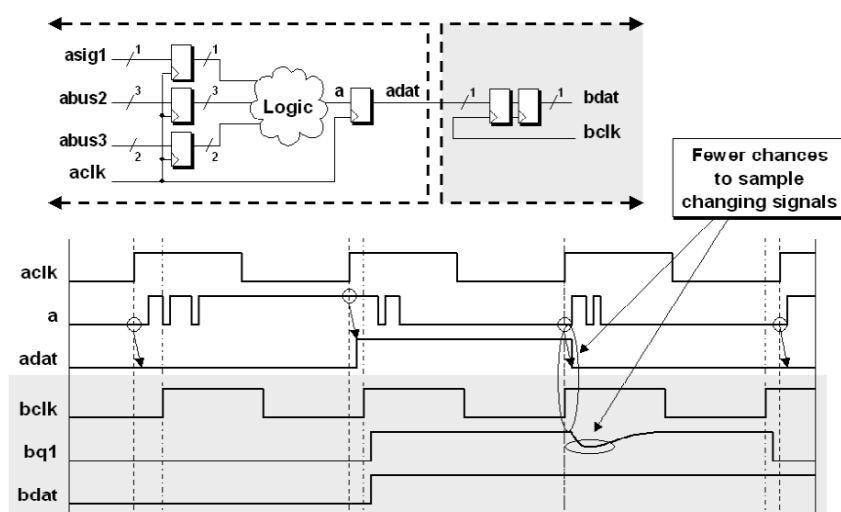
以0.25 μ m工艺为例，参数不变，则DinB的MTBF为9.57*10⁹(years)。因此我们认为两级触发器基本上可以避免亚稳态问题。

🤔有什么问题没有？

1：时钟域A的输出必须经过寄存器！因为组合逻辑电路的输出存在毛刺，信号不稳定，不建议直接输出给其他时钟域。



改正后:

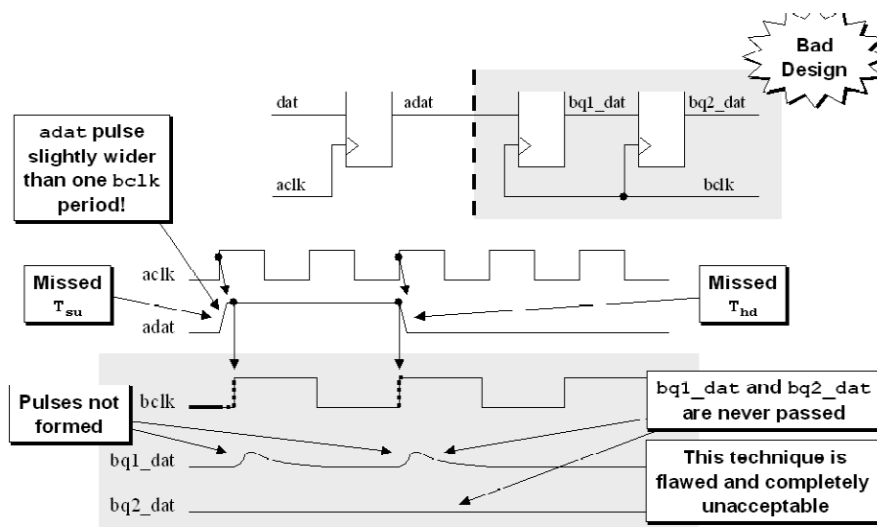


2 三边沿原则！如果较快的时钟域是较慢时钟域频率的1.5倍(或更多)，那较快的时钟信号将采样较慢的CDC信号一次或多次，一般没什么问题。所以对于快时钟域采样慢时钟域信号的情况，使用简单的两个触发器同步器在时钟域之间传递单bit Clock Domain Crossing (CDC)信号即可。

⚠️ 如果快时钟域在慢时钟域频率1倍到1.5倍之间，应该按照慢时钟域采样快时钟域信号的方法去同步。

3 慢到快，有可能在快时钟域多次采样该信号，因此如果是脉冲信号，则需要取边沿(上升沿/下降沿)。

对于慢时钟域采样快时钟域信号的情况，简单的两级同步将不再适用。一般要求在接收时钟域中采样信号要保持三个时钟边沿的时间 ("three edge" requirement)，也就是1.5倍的采样时钟周期。



如果采样信号维持时间过短，则慢时钟域很可能会漏采样。即使采样信号比采样周期略长，也可能面临信号改变正好落在时钟沿的setup和hold violation的区域，如上图。

对于慢时钟域采样快时钟域信号的情况，必须展宽输入信号的长度，使其达到**三边沿原则**。否则只能使用握手协议。

信号展宽需要知道两个频率之间的关系，如果是快时钟域电平信号(一般电平信号持续时间较长，假设满足三边沿原则)，直接两级同步器同步即可。如果快时钟域电平信号不满足三边沿原则，就需要扩展电平宽度。假设快时钟域电平信号是20ns，慢时钟域频率是10MHz(时钟周期是100ns)，那么在这个电平信号进入慢时钟域之前，需要展宽到8个快时钟周期($160\text{ns} > 1.5 \times 100\text{ns}$)，再用两级同步器同步。

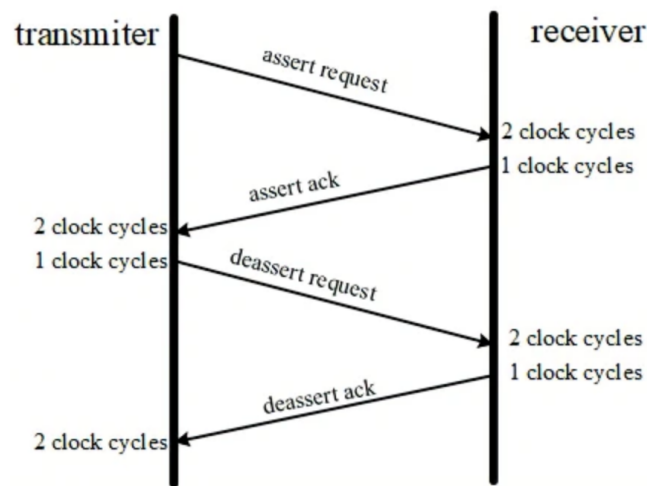
!!! 二级同步器不能用来同步多bit数据!

使用格雷码的话，也只有格雷码自增或自减顺序变化才可以跨时钟传输。

对于多bit数据，在进行传输时候会因为时序问题导致所有寄存器不会同时翻转，所以容易在跨时钟传输的时候出现中间态。使用格雷码可以避免这种现象，但是当格雷码不是按计数顺序变化，这同样是不允许的，因为格雷码每次只有一位发生变化的前提是，数据是递增或递减的。

握手协议：

! 如果不知两者频率关系，则只能用握手来保证正确通信，如下图。



将transmister要传输的请求信号request打一拍送到receiver中，进行两级触发器同步，同步之后的信号ack在反馈回transmister中，在transmister中同样进行两级触发器同步，当transmister接收到同步后的ack信号后，将req信号拉低。

✅：这样的方法是比较稳妥的，可以解决快时钟域向慢时钟域过渡的问题，且其适用的范围很广。

❌：但是会有更多的延时，实现较为复杂，并且需要更多触发器，在对设计性能要求较高的场合应该慎用。

/code示例/

// transmister端

```
always@(posedge clk_a or negedge rst_n)
begin
    if(!rst_n)
        cnt_reg <= 3'd0;
    else if(data_ack_sync1 && !data_ack_sync2 == 1'b1)
        cnt_reg <= 3'd0;
    else if(data_req == 1'b1)
        cnt_reg <= cnt_reg;
    else
        cnt_reg <= cnt_reg + 1'b1;
end
```

//data_ack两级同步

```
always@(posedge clk_a or negedge rst_n)
begin
    if(!rst_n) begin
        data_ack_sync1 <= 1'b0;
        data_ack_sync2 <= 1'b0;
    end else begin
        data_ack_sync1 <= data_ack;
        data_ack_sync2 <= data_ack_sync1;
    end
end
```

//请求接收信号

```
always@(posedge clk_a or negedge rst_n)
begin
    if(!rst_n)
        data_req <= 1'b0;
    else if(cnt_reg == 3'd4)
        data_req <= 1'b1;
    else if(data_ack_sync2 == 1'b1)
        data_req <= 1'b0;
    else
        data_req <= data_req;
end
```

//发送数据

```
always@(posedge clk_a or negedge rst_n)
```

```
begin
  if(!rst_n)
    data <= 4'd0;
  else if(data == 4'd7 && data_ack_sync2 == 1'b1 && data_req == 1'b1 )
    data <= 4'd0;
  else begin
    if(data_ack_sync2 == 1'b1 && data_req == 1'b1 )
      data <= data + 1'b1;
    else
      data <= data;
    end
  end
end

//receiver端
//data_req两级同步
always@(posedge clk_b or negedge rst_n)
begin
  if(!rst_n)
    begin
      data_req_sync1 <= 1'b0;
      data_req_sync2 <= 1'b0;
    end else begin
      data_req_sync1 <= data_req;
      data_req_sync2 <= data_req_sync1;
    end
  end
end

//数据接收确认信号
always@(posedge clk_b or negedge rst_n)
begin
  if(!rst_n)
    data_ack <= 1'b0;
  else if(data_req_sync2 == 1'b1)
    data_ack <= 1'b1;
  else
    data_ack <= 1'b0;
end
end
```

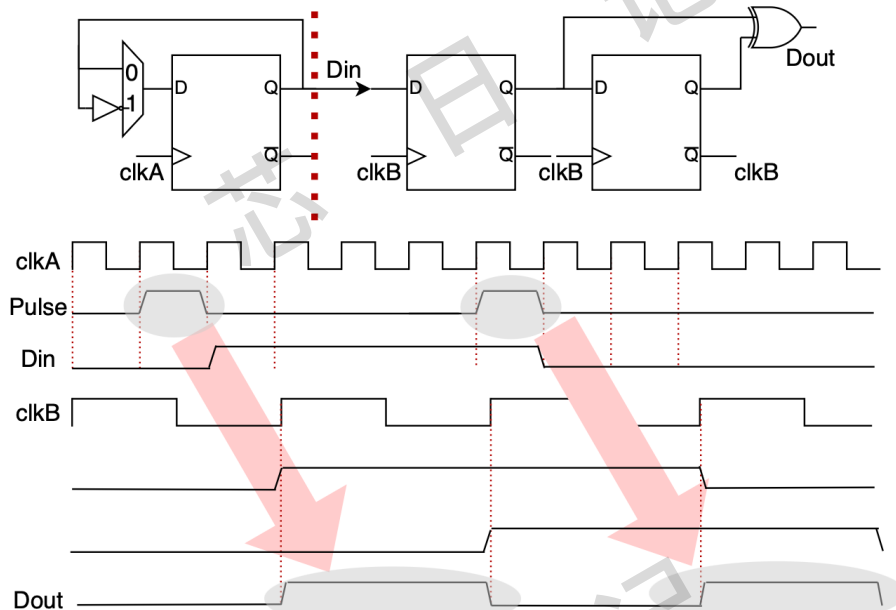
脉冲同步器

同样一个快时钟域的脉冲信号，其宽度只有快时钟域的一个周期的长度，现在需要将该信号同步到一个相对较慢的时钟域，如果要求不能在快时钟域将脉冲信号展宽，并且同样采用双寄存器同步，该如何实现？

对于快时钟域单电平脉冲信号跨时钟到慢时钟域。可以通过翻转电路拉长脉冲电平以保证有效事件被采样，在接收时钟通过边沿检测回复原单电平脉冲。

✓：脉冲同步器在异步时钟域时钟频率彼此差距较大的场景下能节省触发器资源；

✗：但是需要保证发送方两个脉冲之间的间隔足够大，使得生成的信号至少为接收方两个时钟周期。



如果不知道脉冲间隔，快到慢的数据同步一般要使用握手或者异步FIFO来实现。

异步FIFO

常用于多bit数据同步，快到慢、慢到快均适用。

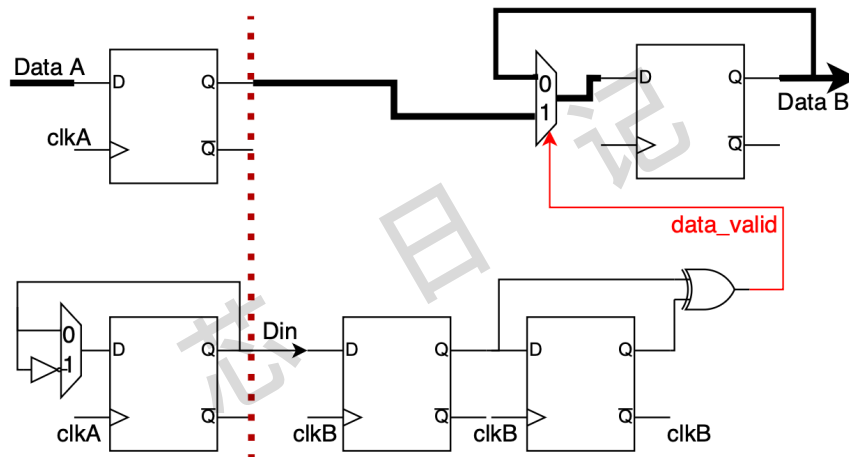
实现细节见之前的笔记！！

DMUX同步

DMUX同步器主要是用于带有数据有效标志信号的多比特数据跨时钟域问题，且多比特数据要保持一段时间。

对数据有效标志信号的处理，就相当于上面提到的单bit脉冲同步电路。

数据有效标志信号在目的时钟clkB下打两拍后就同步于该打拍的时钟域了，此时同步于clkA的数据依然保持有效，将同步后的数据有效标志信号作为多路选择器的选通信号，将数据也同步于clkB时钟域中。如下图，区别就是多了个MUX来选择多bit同步信号。。



一文掌握DC综合流程

综合作用：

- 将行为描述转化为电路结构：把“它应该做什么”的RTL代码，变成“它具体由哪些标准单元组成”的网表。
- 在物理实现之前进行预测和优化：在还没有具体布局布线信息的情况下，基于线负载模型，尽可能地让网表满足时序、面积和功耗目标。

DC所在的位置决定了综合工程师必须同时理解“前因”和“后果”，并做出权衡。

承前：需要深刻理解RTL代码的风格、架构意图和设计约束。

启后：需要预见到后端布局布线可能遇到的问题，并为后端提供一个“干净”且“强健”的起点。

综合的核心难点在于“**多目标优化**”

时序：如何让关键路径的延迟最小，以满足时钟频率要求？

面积：如何在满足时序的前提下，使用最少的芯片面积？

功耗：如何通过插入时钟门控、选择低功耗单元等手段来降低动态和静态功耗？

可测试性：如何在显著影响性能的前提下插入DFT结构？

难点在于，这些目标是相互矛盾的！

- 为了追求时序，你可能需要插入缓冲区、使用驱动能力更大的单元，但这会增加面积和功耗。
- 为了追求面积，你可能要使用最小尺寸的单元，但这会恶化时序。

综合工程师的艺术就在于，如何通过调整综合策略、编译选项、约束的松紧，来为后端找到一个最佳的、可实现的起点。这需要大量的经验和直觉。

而这里面最重要的就是如何约束！SDC文件的质量直接决定了综合结果的质量。约束过紧，会导致

如何设计时序收敛的RTL代码

从项目开始规划架构，有几点⚠️注意事项：

- 将输入和输出结构保持在顶层。
- 将时钟结构保持在顶层，并在独立模块中实现。
- 所有模块的输入和输出采用寄存器架构。解决模块内的时序问题比跨模块的路径问题更容易。寄存器型 I/O 的另一个好处是，它可以防止逻辑在优化期间在模块之间移动。
- 确保接口尽可能利用标准接口，例如 AXI、AXI Lite 和 AXI Stream。

合理的时钟域规划

- A. 关键路径在同频同相时钟域内，确保关键数据路径处于同一个时钟域，避免复杂的跨时钟域处理成为时序瓶颈。
- B. 异步时钟域隔离：明确划分异步时钟域，并使用成熟的同步器（如两级触发器）处理跨时钟域信号。这能将时序问题隔离在单个时钟域内解决。
- C. 谨慎使用门控时钟：使用标准单元库提供的集成时钟门控单元。

模块化与流水线设计

- A. 模块划分：按功能和数据流划分模块，并定义清晰的接口。模块内部尽量是同步设计，时序问题在模块内部解决。
- B. 插入流水线：这是解决长组合逻辑路径的最有效方法。
识别关键路径：预估或通过早期综合识别组合逻辑延迟过长的路径。

插入寄存器：将长的组合逻辑链切断，插入寄存器级（Pipeline Stage）。这将一个时钟周期的工作分摊到多个周期，从而提高了整体时钟频率。

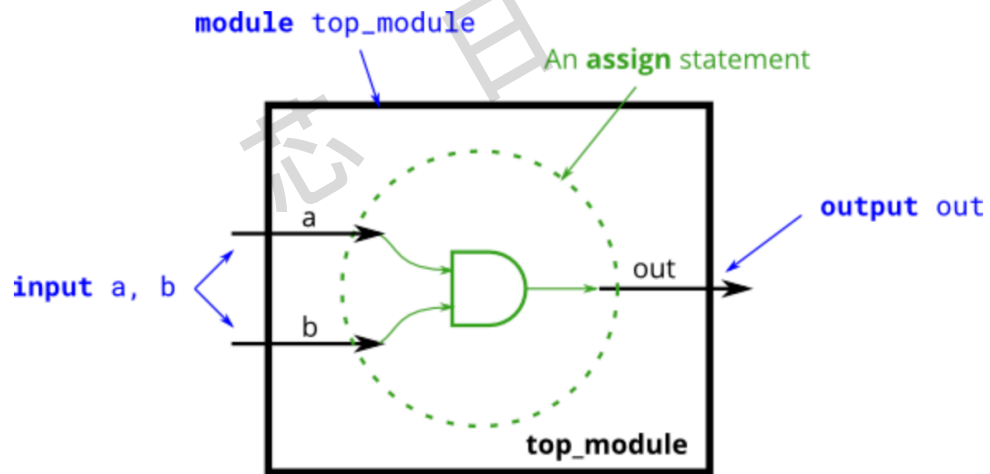
平衡流水线级数：确保各级流水线的延迟大致相等，避免出现新的关键路径。

Coding技巧

verilog 电路

任何verilog语句都可以拆成组合逻辑和时序逻辑。

先看一个简单的电路：assign out = a&b；这个语句估计每个ICer都能画出来。



同理或门，非门一样。与或非门就是数字电路最基本的组合逻辑单元，任何所有复杂的算法或者公